

UC Riverside

UC Riverside Previously Published Works

Title

DECAF: A Platform-Neutral Whole-System Dynamic Binary Analysis Platform

Permalink

<https://escholarship.org/uc/item/3x9798xm>

Journal

IEEE Transactions on Software Engineering, 43(2)

ISSN

0098-5589

Authors

Henderson, Andrew
Yan, Lok Kwong
Hu, Xunchao
[et al.](#)

Publication Date

2017

DOI

10.1109/tse.2016.2589242

Peer reviewed

DECAF: A Platform-Neutral Whole-System Dynamic Binary Analysis Platform

Andrew Henderson, *Senior Member, IEEE*, Lok Kwong Yan, *Senior Member, IEEE*, Xunchao Hu, Aravind Prakash, Heng Yin, *Member, IEEE*, and Stephen McCamant, *Member, IEEE*

Abstract—Dynamic binary analysis is a prevalent and indispensable technique in program analysis. While several dynamic binary analysis tools and frameworks have been proposed, all suffer from one or more of: prohibitive performance degradation, a semantic gap between the analysis code and the program being analyzed, architecture/OS specificity, being user-mode only, and lacking APIs. We present DECAF, a virtual machine based, multi-target, whole-system dynamic binary analysis framework built on top of QEMU. DECAF provides Just-In-Time Virtual Machine Introspection and a plugin architecture with a simple-to-use event-driven programming interface. DECAF implements a new instruction-level taint tracking engine at bit granularity, which exercises fine control over the QEMU Tiny Code Generator (TCG) intermediate representation to accomplish on-the-fly optimizations while ensuring that the taint propagation is sound and highly precise. We perform a formal analysis of DECAF's taint propagation rules to verify that most instructions introduce neither false positives nor false negatives. We also present three platform-neutral plugins—Instruction Tracer, Keylogger Detector, and API Tracer, to demonstrate the ease of use and effectiveness of DECAF in writing cross-platform and system-wide analysis tools. Implementation of DECAF consists of 9,550 lines of C++ code and 10,270 lines of C code and we evaluate DECAF using CPU2006 SPEC benchmarks and show average overhead of 605 percent for system wide tainting and 12 percent for VMI.

Index Terms—Dynamic binary analysis, dynamic taint analysis, virtual machine introspection

1 INTRODUCTION

DYNAMIC binary analysis has demonstrated its strength in many research problems, such as malware analysis, protocol reverse engineering, vulnerability signature generation, software testing, and profiling and performance optimization. There are many analysis platforms for process-level binary instrumentation, such as Pin [1] and Valgrind [2].

Compared to process-level program instrumentation and analysis, whole-system dynamic binary analysis has unique advantages. First, it provides a full system view, including the OS kernel and all running applications, allowing the analysis of kernel activity and the interactions among multiple user-space processes. Second, the code instrumentation and analysis are performed from entirely outside of the context of the guest system under analysis (typically executing within a virtual machine (VM)). In contrast, user-level instrumentation tools share the same memory space as the instrumented program execution. Leveraging virtualization techniques, whole-system dynamic binary analysis provides better transparency and stronger isolation than that of process-level

instrumentation tools. This is especially important within the context of analyzing malicious code that attempts to detect, evade, and tamper with the analysis environment.

Although much research has been performed to make use of whole-system dynamic binary analysis to solve various security problems [3], [4], [5], [6], [7], [8], little attention has been paid to the analysis framework itself. Such tools are often tailored to solve specific problems in an ad-hoc manner. Many times, analysts must still develop new analysis tools from scratch to meet their own specific needs.

Building a generic, whole-system dynamic binary analysis platform that suits various needs is desirable, but challenging. For example, previous work in this area, TEMU [9] in the BitBlaze binary analysis toolkit [10], provides a rich set of capabilities and has facilitated many binary analysis research projects. Its design, while feature-rich, creates execution overhead that may be far too heavyweight for simpler analyses that do not require all of TEMU's features.

In this paper,¹ we present *DECAF*,² a new whole-system dynamic binary analysis platform built upon the QEMU system emulator. DECAF aims to “Make It Work, Make It Right, Make It Fast”. This means that DECAF must not only provide the same set of capabilities as analysis systems such as TEMU, but it must also follow proper principles in its design. DECAF offers analysis results of better quality, and with a higher correctness guarantee, than TEMU while still conducting analyses more efficiently. Particularly, we

- A. Henderson, X. Hu, A. Prakash, and H. Yin are with the Department of Electrical Engineering and Computer Science, Syracuse University, Syracuse, NY 13244. E-mail: {anhender, xhu31, arprakas, heyin}@syr.edu.
- L. Yan is with the Department of RIG, Rome Laboratory, Rome, NY 13441. E-mail: lok.yan@us.af.mil.
- S. McCamant is with the Department of Computer Science & Engineering, University of Minnesota (Twin Cities), Minneapolis, MN 55455. E-mail: mccamant@cs.umn.edu.

Manuscript received 28 Dec. 2014; revised 5 May 2016; accepted 24 May 2016. Date of publication 6 July 2016; date of current version 21 Feb. 2017.

Recommended for acceptance by A. Tanter.

For information on obtaining reprints of this article, please send e-mail to: reprints@ieee.org, and reference the Digital Object Identifier below.

Digital Object Identifier no. 10.1109/TSE.2016.2589242

1. Approved for Public Release; Distribution Unlimited : 88ABW-2016-3163, 27 June 2016.

2. DECAF stands for Dynamic Executable Code Analysis Framework.

overcome the following key challenges in building a whole-system dynamic binary analysis platform:

1. *How to Reconstruct a Fresh OS-Level Semantic View from Completely Outside of the Guest System?* As we run a virtual machine inside a whole-system binary analysis framework and perform various analysis tasks from outside, we must reconstruct the OS-level semantic view of the guest VM from outside, known as Virtual Machine Introspection (VMI). Several efforts (such as VMWatcher [11], Virtuoso [12], and VMST [13]) have been made to bridge this *semantic gap* and reconstruct the OS-level semantic view. However, the question of “when to reconstruct” has not been addressed. In a running system, the OS-level semantic views constantly change (e.g., a process starts or terminates, a code module is loaded or unloaded). For dynamic analysis, we must be aware of these new events “just-in-time” at the moment they occur. TEMU circumvented this problem by inserting a kernel module into the guest OS within the VM. This kernel module hooks several system events, retrieves OS-level information, and passes it to the hypervisor through a spare port. This circumvention clearly violates the external monitoring principle for VMI, and it can be easily subverted by the malicious code inside the VM.

In DECAF, we propose a new, novel solution to reconstructing a fresh OS-level semantic view by only monitoring hardware-level events. Such an approach has not, to our knowledge, been proposed before. It provides notification of OS-level events without requiring the expensive polling of guest kernel data structures or the violation of the external monitoring principle.

2. *How to Provide an Event-Based Programming Paradigm That is Both Correct and Efficient?* Most of the existing analysis platforms provide instrumentation interfaces only, through which a plugin can specify which instructions to instrument and what instrumentation code should be run. While this instrumentation approach is simple and flexible, it places a burden on the plugin developers to decide exactly how to instrument guest program execution. Such an approach is acceptable for user-level instrumentation, but it becomes difficult within a whole-system setting. Properly instrumenting whole-system execution requires the analyst to be familiar with the low-level system details of the guest system, such as exceptions, interrupts, page faults, context switches, etc.

Therefore, DECAF must provide an event-based interface, through which an analyst can register for events in various selected contexts (e.g., a process, the kernel space, or a kernel module). Under the hood, DECAF takes care of what instrumentation code to selectively insert and where, and it ensures that the inserted instrumentation code is correct and efficient. TEMU provides a similar high-level interface, but achieves it in a naive way: it inserts instrumentation code uniformly in *all* translated code blocks and decides at execution time whether to deliver the events to the plugin. This guarantees the correctness of event

processing, but incurs unnecessarily high runtime overhead. DECAF selectively inserts instrumentation into only the code blocks where it is needed, dramatically lowering overhead and improving performance.

3. *How to Implement Precise, Sound, and Lossless Tainting?* Dynamic taint analysis (tainting) is a powerful dynamic binary analysis technique. Many taint system implementations exist [10], [14], [15], [16], [17]. Among these implementations, two important factors are often overlooked. First, most of these implementations are not precise enough (resulting in overtainting), and some of them are not even sound (resulting in undertainting). This means that these taint analysis systems would unnecessarily mark many memory locations as tainted and/or fail to taint certain memory locations and CPU registers that should be tainted. When dealing with security problems, an unsound implementation may miss real attacks, while an imprecise implementation may raise too many false alarms.

Second, we often need to track tainted data originating from multiple taint sources by applying multiple labels. Many taint analysis implementations do not distinguish among multiple taint labels. For the ones that do, they do not provide a lossless guarantee. Each tainted byte or word is associated with up to a small number of taint labels, due to space constraints on shadow memory. When a memory location or CPU register is tainted from more taint sources than those that can be kept in the shadow memory, the remaining are *lost*!

To achieve high precision, DECAF maintains taint information for every bit of registers and memory locations, and it applies precise tainting rules for most instructions at the QEMU Tiny Code Generator (TCG) intermediate representation (IR) level. Getting these per-instruction rules right is important for soundness and precision, so we perform a careful analysis using definitions based on bit-level non-interference. We examine the information-flow patterns in integer operations experimentally, survey previous systems, and in several cases design new propagation rules when no previous rule was sound and precise. We verify the soundness and precision of these best rules for each operation using two decision procedures (automatic theorem provers), and also the complete implementation using a new technique called *per-trace verification*. This allows DECAF to offer what we believe is the first sound implementation of whole-system tainting.

To support any number of taint labels without the information loss seen in other systems, DECAF separates tracking of taint status from tracking taint labels. Taint status is tracked efficiently and inline during execution, while taint labels are tracked in an asynchronous manner via plugin-based logging.

4. *How to Provide Strong Support for Cross-Platform Analysis?* Ideally, we want to have the same analysis code (with minimum platform-specific code) to work for different guest CPU architectures (e.g., x86 and ARM) and different guest operating systems (e.g., Windows and Linux). This requires the analysis framework to hide the guest architecture- and OS-

specific details from the analysis plugins. Further, to make the analysis framework maintainable and easily extensible to new architectures and OSes, the platform-specific code within the framework must be minimized. Some instrumentation tools, like Pin [1], can run in both Linux and Windows, but, until now, no analysis tool provides support for both multiple architectures and multiple OSes. DECAF provides support for multiple platforms by implementing core instrumentation and analysis tasks at the TCG IR level, independent of the CPU architecture of the VM. DECAF's plugin API is engineered to hide many architecture and OS specific details.

DECAF is an open-source project [18]. Since the release of its first version in January 2013, it has received over 5,000 downloads and has been utilized in a number of malware and security analysis studies [19], [20], [21], [22]. A handful of analysis plugins have also been built on top of it to demonstrate the power of this framework. We showcase three plugins to demonstrate how DECAF solves various binary analysis problems. By hooking the entries and exits of API functions specified in a configuration file, *API Tracer* is able to trace the API invocations of a specified process and the processes spawned from it. *Keylogger Detector* tracks tainted keystrokes propagating throughout the OS kernel and across user-level processes to detect keyloggers. *Instruction Tracer* logs instructions executing within a specified context (such as a user-level process or kernel module). These plugins are mostly platform neutral. Since DECAF provides a platform-independent programming interface, these plugins can analyze binary executables for multiple hardware architectures (including x86 and ARM) and multiple OSes (including Windows and Linux), requiring no, or very minimal, platform-specific code.

2 SYSTEM OVERVIEW

DECAF is built on top of QEMU [23], the whole-system emulator and dynamic translator. By extending QEMU, DECAF inherits a mature and feature-rich platform to use as a starting point when implementing its instrumentation and analysis functionality.

2.1 Understanding QEMU

QEMU's whole-system emulator functionality acts as a type-2 hypervisor for executing guest virtual machines. It makes use of dynamic binary translation techniques to emulate multiple target guest architectures, so the architecture of the guest environment can differ from that of the host machine. Virtual guest hardware devices, such as network interfaces and IDE/SCSI controllers, are implemented in software and pass data through to the devices physically present on the host system as needed. Because all aspects of the guest environment (e.g., CPU, RAM, hardware devices) are emulated in software, DECAF has many opportunities to monitor the runtime behavior of the guest system.

QEMU decouples the specific details of the guest CPU from that of the host using its Tiny Code Generator. TCG translates the instructions of the guest environment into an intermediary representation (IR) of architecture-neutral set

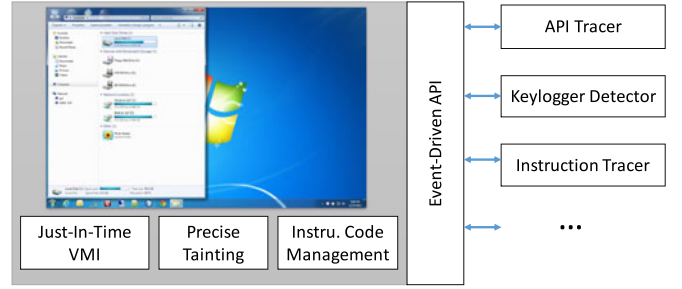


Fig. 1. The overview of DECAF.

of RISC-like instructions. These instructions include common ALU operations (e.g., add, sub, xor), memory load/store, and control flow transfer. This IR is then dynamically translated into the native instructions of the host system and executed.

DECAF modifies QEMU's TCG to selectively insert instrumentation into the IR at the point of guest-to-IR translation. At the point of IR-to-host translation, the instrumentation becomes embedded within the host instruction stream without disturbing the semantic meaning of the guest's execution. This enables DECAF to support the analysis of a wide variety of different guest architectures while requiring only a minimal amount of architecture-specific code. This process is detailed in Section 3.

2.2 DECAF Components

Fig. 1 provides an overview of DECAF. Inside the virtual machine, we can run the programs of interest and conduct various analyses externally via analysis plugins. DECAF has the following key components:

Just-in-Time VMI. This VMI component is able to reconstruct a fresh OS-level view of guest executing within the virtual machine, including its processes, threads, code modules, and symbols, to support binary analysis. Further, to support multiple architectures and operating systems, it follows a platform-neutral design principle. The workflow for extracting OS-level semantic information is common across multiple architectures and OSes. The only platform-specific handling lies in what guest kernel data structures and what fields to extract information from. We present more details about VMI in Section 4.

Precise, Lossless Dynamic Taint Analysis. DECAF ensures precise tainting by maintaining bit-level taint precision for CPU registers and memory, and inlining precise tainting rules in the translated code blocks. Thus, the taint status of every CPU register and memory location is processed and updated synchronously during the code execution of the virtual machine. The propagation of taint labels is done by recording to a taint propagation log via a plugin. Later, this log can be analyzed to determine label propagation. This label analysis is done in an asynchronous manner for two reasons: 1) it is impractical and expensive to maintain an unlimited number of labels for each tainted bit in the shadow memory; and 2) for most taint analysis problems, we do not need to know which taint labels are associated with all tainted bits in real time. We are only interested in a small amount of key data (e.g., the x86 EIP register or a sensitive memory buffer), and when they become tainted. We can then trace back through the taint propagation log and retrieve their labels. By


```

/* Define some globals for our plugin logic. */
static plugin_interface_t my_interface;
static DECAF_Handle handle_keystroke_cb;
static DECAF_Handle handle_read_taint_mem_cb;
static int taint_key_enabled = 0;

/* Define the callback triggered when tainted memory
   is read. */
static void
my_read_taint_mem_cb(DECAF_Callback_Params *param) {
    char name[128];
    tmodinfo_t tm;
    /* Find the code module accessing tainted memory. */
    if ( VMI_locate_module_c( DECAF_getPC(cpu_single_env),
        DECAF_getPGD(cpu_single_env), name, &tm) == 0 )
        /* Virtual address and module of the access. */
        DECAF_printf("INSN 0x%08x, Module '%s' Read-Key\n",
            DECAF_getPC(cpu_single_env), tm.name);
}

/* Define the callback triggered when a keystroke is
   entered into the guest via a QEMU monitor command. */
static void
my_sendkey_cb(DECAF_Callback_Params *params) {
    *params->ks.taint_mark = taint_key_enabled;
    taint_key_enabled = 0;
    DECAF_printf("taint_key%d\n", params->ks.keycode);
}

/* Define the function called when the plugin-specific
   "taint_sendkey" QEMU monitor command is used. */
static void
do_taint_sendkey(Monitor *mon, const QDict *qdict) {
    if (qdict_haskey(qdict, "key")) {
        /* Enable tainting for the next keystroke */
        taint_key_enabled = 1;
        /* Send the tainted keystroke into the guest */
        do_send_key(qdict_get_str(qdict, "key"));
    }
}

/* Define the "taint_sendkey" QEMU monitor command. */
static mon_cmd_t my_term_cmds[] = {
    {
        .name = "taint_sendkey",
        .args_type = "key:s",
        .mhandler.cmd = do_taint_sendkey,
        .params = "taint_sendkey_key",
        .help = "Send a tainted keypress to the guest",
    },
    {NULL, NULL, },
};

/* Define a cleanup function for plugin unload. */
static void
my_cleanup(void) { /* Perform cleanup here. */ }

/* This is executed upon loading this plugin. */
plugin_interface_t *
init_plugin(void) {
    /* Register plugin-specific QEMU monitor commands. */
    my_interface.mon_cmds = my_term_cmds;
    /* Register cleanup function called at plugin unload. */
    my_interface.plugin_cleanup = my_cleanup;
    /* Register for DECAF callback events. */
    handle_read_taint_mem_cb = DECAF_register_callback(
        DECAF_READ_TAINTMEM_CB, my_read_taint_mem_cb, NULL);
    handle_keystroke_cb = DECAF_register_callback(
        DECAF_KEYSTROKE_CB, my_sendkey_cb, NULL);
    /* Done! Return this new plugin interface to DECAF. */
    return &my_interface;
}

```

Fig. 2. A sample plugin that keeps track of tainted keystrokes.

implementing such a tainting logic mainly in QEMU's intermediate representation level (more concretely, at the TCG instruction level), it becomes much simpler to extend tainting support to a new CPU architecture. Section 5 provides more details about our taint analysis implementation.

Event-Driven Programming Interface. Compared to many existing analysis frameworks [1], [2] that provide only an instrumentation interface, DECAF provides an event-driven

programming interface. It means that DECAF's design of "instrument in the translation phase and then analyze in the execution phase" is invisible to the analysis plugins. Plugins only need to register for specific events and implement the corresponding event handling functions. The details of how the code is instrumented are handled by the framework, not by the plugins. Such details include how to generate the instrumentation code for inserting these event handlers into the translated code stream and how to maintain instrumentation code consistency when new event handlers are registered and old ones are removed.

Dynamic Instrumentation Management. To reduce runtime overhead, the instrumentation code is inserted into the translated code only where necessary. For example, when a plugin registers a function hook for a function's entry point, the instrumentation code for this hook is only placed once (at the function entry point). When the plugin unregisters this function hook, the instrumentation code will also be removed from the translated code accordingly. To ease the development of plugins, the management of dynamic code instrumentation is completely taken care of in the framework, and thus invisible to the plugins.

2.3 Example DECAF Plugin

Fig. 2 presents the source code for an example DECAF plugin that detects keylogger malware within the guest system. This plugin tracks the propagation of tainted keystrokes throughout the entire guest environment, and it is both guest architecture and OS independent. The same plugin code works for x86 and ARM, Windows and Linux. Whenever possible, DECAF provides generic functions to abstract away any architecture-dependent details of the guest. For example, `DECAF_getPC` will return the program counter (e.g., EIP in x86 and R15 in ARM), and `DECAF_getPGD` will return the page table directory (e.g., CR3 in x86 and CP15 in ARM).

DECAF plugins work by registering callback functions that are executed when events of interest occur within the guest. The sample plugin defines two functions, `my_read_taint_mem_cb` and `my_sendkey_cb`, that will be registered as callback functions. `my_read_taint_mem_cb` will be called whenever tainted guest memory is read (the `DECAF_READ_TAINTMEM_CB` event). `my_sendkey_cb` will be called whenever a tainted keystroke is entered into the system (the `DECAF_KEYSTROKE_CB` event).

Because it is often necessary for an analyst to interact with a plugin during guest execution, DECAF leverages the QEMU command monitor. The monitor is a shell that accepts commands for controlling and querying the runtime behavior of QEMU, such as starting/stopping guest execution, saving the state of the VM, and profiling QEMU's resource usage. The example plugin code specifies a plugin-specific monitor command, `taint_sendkey`, in the `my_term_cmds[]` array. When this command is entered into the QEMU monitor, the plugin's `do_taint_sendkey` function is called and a tainted keypress is entered into the guest VM. The `taint_sendkey` command is only available while the plugin is loaded. Upon unloading the plugin, any plugin-specific commands are removed from the monitor.

Every plugin must have an `init_plugin` function. This function is called to initialize the plugin and return a pointer

```
// Start of translation block
// Original instruction: orl %ebx, %eax
mov_i32 tmp11, ebx
mov_i32 tmp12, eax
or_i32 tmp13, tmp12, tmp11
// Original instruction: addl $0x01, %eax
mov_i32 tmp14, $0x01
add_i32 tmp15, tmp14, tmp13
mov_i32 eax, tmp15
// End of translation block
goto_tb $0x0
```

(a)

```
// Start of translation block
// Insert DECAF_BLOCK_BEGIN callback
mov_i32 tmp21, $<CURRENT_ADDRESS>
mov_i32 tmp22, $DECAF_invoke_block_begin_callback
call tmp22, $0x0, $0, env, tmp21
// Original instruction: orl %ebx, %eax
// Insert DECAF_INSN_BEGIN callback
mov_i32 tmp23, $DECAF_invoke_insn_begin_callback
call tmp23, $0x0, $0, env
mov_i32 tmp11, ebx
mov_i32 tmp12, eax
or_i32 tmp13, tmp12, tmp11
// Insert DECAF_INSN_END callback
mov_i32 tmp24, $DECAF_invoke_insn_end_callback
call tmp24, $0x0, $0, env
// Original instruction: addl $0x01, %eax
// Insert DECAF_INSN_BEGIN callback
mov_i32 tmp25, $DECAF_invoke_insn_begin_callback
call tmp25, $0x0, $0, env
mov_i32 tmp14, $0x01
add_i32 tmp15, tmp14, tmp13
mov_i32 eax, tmp15
// Insert DECAF_INSN_END callback
mov_i32 tmp26, $DECAF_invoke_insn_end_callback
call tmp26, $0x0, $0, env
// End of translation block
// Insert DECAF_BLOCK_END callback
mov_i32 tmp27, $DECAF_invoke_block_end_callback
call tmp27, $0x0, $0, env
goto_tb $0x0
```

(b)

Fig. 3. DECAF inserts instruction execution callbacks into the original TCG code stream (a) to create an instrumented opcode stream (b) to trigger helper function calls to plugin callback functions.

to a `plugin_interface_t` structure, which specifies any plugin-specific monitor commands and a cleanup function (`my_cleanup` in the sample plugin) to be called when the plugin is unloaded. The `init_plugin` function typically registers callback functions for any guest events of interest, but registering and unregistering callbacks can be performed at any point after the plugin has been loaded.

When the analyst loads this sample plugin and then enters the `taint_sendkey` command into the monitor, the registered callback `my_send_keystroke` is called and the corresponding keystroke is tainted. Thereafter, the tainted keystroke will propagate from the keyboard device, through the OS kernel, and to the destination user-level program. Since DECAF performs whole-system dynamic taint analysis, we are able to observe this entire taint propagation flow. Whenever an instruction reads a tainted memory location, the framework will call the registered `my_read_tainted_mem` callback, which checks the code module in which this instruction is located. The relevant information about this taint event is then logged for offline analysis.

3 SELECTIVE CODE INSTRUMENTATION

To meet the requirements of efficiency and cross-platform for code instrumentation, DECAF selectively inserts instrumentation code at QEMU's intermediate representation level.

Dynamic Binary Translation in QEMU. To support multiple architectures, QEMU makes use of a compiler backend, called Tiny Code Generator, as its dynamic binary translation engine. QEMU translates each basic block of guest instructions into a series of architecture-independent TCG instructions grouped together as a TCG translation block (TB). The TCG compiler then translates each TB into a piece

of native code to be executed on the host. Fig. 3a shows how two x86 instructions are translated into TCG instructions.

TCG instructions include common ALU operations (e.g., add, sub, xor), memory load/store, and control flow transfer. The parameters for each TCG instruction can be temporary variables (registers that exist only within the scope of the current TB), global variables, and constants. For more complex, guest-specific instructions (e.g., floating point operations), a `call` TCG instruction exists for making calls to high-level language helper functions that implement the complex functionality. In this manner, TCG cleanly decouples specific details of the guest from that of the host.

Placement of Code Execution Events. Our code instrumentation must work coherently with the TCG-based dynamic binary translation process. Events like “block begin/end” and “instruction begin/end” are used for tracing program execution. When callbacks for these events are registered by a plugin, DECAF inserts the proper helper function calls into the necessary TBs by pausing the guest's execution, flushing the necessary TBs, retranslating those TBs to include calls to the helper functions, and then resuming the guest's execution. Since callbacks are triggered inline with the guest's execution, they are synchronized to the occurrence of events of interest.

Fig. 3b shows that the two helper functions `DECAF_invoke_insn_begin_callback` and `DECAF_invoke_insn_end_callback` are inserted at the beginning and the end of each guest instruction. For many analyses, we are only interested in the execution of a small portion of the system, such as a single kernel module or user-level process. Plugins can specify ranges of memory addresses, or even a single address, of interest when registering for callbacks. Callback helper functions are only placed into the necessary TBs, and only at the proper locations within each TB, to capture these events as they occur. This greatly reduces the runtime overhead of DECAF.

An important design decision here is a dispatch mechanism. For each kind of event (e.g., “block begin”), we only insert a single helper function (e.g., `DECAF_invoke_block_begin_callback`) at each desired program location, and within the helper function, we will iterate through all the registered callbacks for that event and decide which callbacks to trigger. There are two important reasons: avoiding multiple callbacks at the same location and efficiently removing stale instrumentation code.

The plugins and the platform itself may altogether register multiple callbacks on the same event. A dispatch mechanism like this can avoid inlining repeated helper function calls, which negatively impacts the performance. More importantly, in this whole-system emulator, the callback functions inserted into the code stream are executed within the context of the entire guest system. For example, instrumentation code inserted into a shared library will be executed in all processes with this library loaded. So, we need the dispatch mechanism to decide at execution time if the current execution context is the correct one for each registered callback.

We also need a mechanism to nicely remove any stale instrumentation code. A plugin may frequently register and unregister callbacks at runtime. A common example is function hooking. A plugin may need to examine the return

value and output parameters when an API call returns. To do so, the plugin registers a hook on the entrypoint of that call. When that hook is invoked, the plugin retrieves the return address of the API call and registers a second hook on its return address. When the second hook is invoked, the plugin can inspect the return value and output parameters. After that, the plugin removes the second hook for efficiency. Thanks to the dispatch mechanism described above, we no longer have to immediately remove the second hook, which involves flushing the corresponding code cache and forcing a retranslation, which hurts runtime performance. If no callbacks are associated with an inserted helper function, then no callbacks will be dispatched, which is expected. This little extra function call overhead is several magnitudes smaller than frequent code cache flushing and retranslation. Therefore, we postpone the actual code cache flush to a much later time to improve efficiency.

MMU, IO, and Higher-Level Events. Events like “memory read/write” and “tainted memory read/write” are related to the Software Memory Management Unit (in short, SOFTMMU) in QEMU. QEMU must translate each guest virtual address into a guest physical address, and then translate that into a host virtual address. Therefore, the instrumentation for MMU-related events is straightforward: the helper functions are directly inserted into the SOFTMMU code. Of course, a dispatch mechanism is still needed to properly deliver the callbacks to the plugin. Some higher-level events are derived from these low-level memory events. For example, VMI events (such as process creation and deletion) are derived from the TLB execute miss event.

QEMU emulates a set of common IO devices, such as hard disks, keyboards, and network cards. We can easily instrument the IO events related to these devices by inserting helper functions inside each virtual device’s implementation.

Dynamic Tainting Control. A unique feature of DECAF is that it can dynamically turn tainting on and off during analysis. This is a particularly important feature for a whole-system analysis framework. Due to the considerable runtime overhead of tainting, we would like to enable tainting only when needed. When a user or plugin requests to switch tainting on or off, DECAF will flush the entire translation code cache and instrument the new code blocks under the new settings. Details of the implementation of tainting instrumentation at the TCG-instruction level are explained in Section 5.

4 JUST-IN-TIME VMI

As a binary analysis platform, DECAF must reconstruct the following OS-level semantics of the guest to facilitate custom analysis tasks “out of the box”: (1) *Processes*. We need to know what processes are running in the VM. As many analysis tasks only focus on one or very few user-level processes, this process information is essential to limit the amount of added instrumentation. (2) *Threads*. Many programs are multi-threaded. Knowing which threads are running within a given process is also important for many analysis tasks. (3) *Code Modules*. Within a process’s memory space, a main executable and several shared libraries are loaded. Binary analysis often needs to know which code module an instruction comes from. Thus, this code module information is also required. (4) *Exported Symbols*. Shared libraries export a list of functions to enable other code modules to dynamically link with each

other and call exported functions by name. Retrieving exported symbols greatly helps in understanding a program’s behavior at the API level, as APIs are exported symbols.

4.1 Goals and Challenges

We have three primary design goals for DECAF’s just-in-time VMI. First, we must always have a fresh view of the guest OS. For many analysis tasks, we must be immediately notified when a new process is created or a new code module is loaded so that we can observe a program’s complete execution from beginning to end. No existing VMI techniques provide such a strong timing guarantee.

Second, our VMI technique must be as platform-independent as possible, as the same techniques should work for different CPU architectures and different OSes with minimal platform-specific handling. Note that to meet our first design goal, one could hook specific system calls (e.g., fork and exec) or kernel functions. However, this approach is very OS-specific and often changes across different OS versions, so it would fail to meet our second design goal.

Third, as VMI is a basic functionality required by almost every analysis plugin, the performance overhead for our VMI technique must be minimal. A key challenge is to meet both this performance requirement and the strong timing guarantee of our first goal simultaneously. We must monitor certain system events more frequently, which may incur high runtime overhead, to continually maintain a fresh view of the guest OS.

4.2 Solution

We rely upon the following three observations that commonly hold true across modern platforms to achieve our goals for just-in-time VMI. First, each process must have its own memory space and each CPU architecture must have a register to indicate the current base address of the memory space of that process (e.g., CR3 in x86 and CP15 in ARM). DECAF uses this register to uniquely identify each new process. Second, a Translation Look-aside Buffer (TLB) will have an “execute” cache miss whenever a new code page is loaded and executed. Third, upon context switch, the old mappings in the TLB will be flushed. Therefore, whenever a new process is created or a new module is loaded, DECAF’s VMI captures the exact moment it occurs via a TLB Execute cache miss.

The usage of TLB Execute cache misses for VMI is a novel contribution of the DECAF system. Process-level VMI approaches do not have visibility of such hardware events, but they generally have no need to observe them because the semantics of the process under analysis are already well-known. Whole-system VMI approaches must either continually watch key kernel data structures for changes or violate the external monitoring principle by placing notification code within the guest kernel (using a custom kernel driver or module). Monitoring cache misses allows DECAF to eliminate the overhead of polling key data structures while not violating the external monitoring principle. This results in lower VMI overhead when executing guest environments.

Fig. 4 illustrates the VMI workflow. Whenever we observe a TLB Execute cache miss, we first check whether the current program counter is in the kernel space. If not, we will determine if the current process is newly created by searching for the current PGD in the process list. If we cannot find it, this

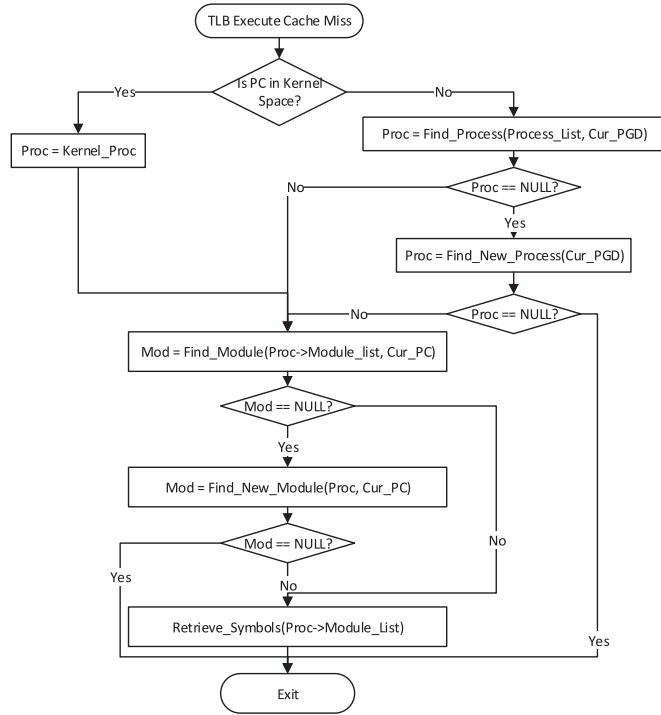


Fig. 4. The VMI flowchart.

process must be new, so we traverse the kernel data structures (i.e., active process list) of the guest to retrieve information about the newly created process. We only traverse kernel data structures (which can be a costly operation) when there is a new process. Checking for the presence of existing processes in the hash table takes constant time.

After we locate the right process (either it already exists or is newly created), we check if a new code module is loaded. Again, we use a hash table to quickly determine whether the current program counter falls into any code modules that have been loaded into the current process memory space. If not, we have found a new code module and will traverse the module list in the guest to retrieve information (such as module name, base address, size) about the new module.

Once we locate the current code module, we start retrieving the exported symbols of the code modules directly from memory. We must parse the headers (PE for Windows, and ELF for Linux) of each code module to extract symbols. Note that we may not be able to completely retrieve symbols for a newly loaded module the first time we see it, as related pages may not yet be loaded into memory. Therefore, on future TLB Execute misses, we will recheck this code module to see if the symbols are now available for retrieval.

The symbol extraction process is fairly heavyweight because it requires many memory reads from the guest to parse executable headers and copy the symbols. However, we only need to do it once for each code module across all guest processes. Since most code modules are shared libraries (.so files in Linux and .dll files in Windows), this overhead is amortized across the creation of multiple processes.

TLB cache misses cannot help us find the exact moment when a process has terminated or a module has been unloaded. To find such events, we must periodically traverse the kernel data structures to find deleted process objects and unloaded code modules. In general, these events are not so

timing critical for binary analysis purposes, unlike process creation and module loading events. So, periodically checking (e.g., every 1 or 5 seconds) is acceptable. If an analyst must know the precise time when such termination events happen, the plugins must implement their own mechanism to do so, such as hooking specific functions in the guest execution.

This VMI workflow avoids inserting OS-specific hooks into the VM to obtain a fresh view of the guest OS, and it also avoids frequent memory reads in the VM. The only platform-specific knowledge for this VMI workflow is what kernel data structures to examine and how to interpret the related fields in those structures. The definition of these data structures are publically available. Compared to hooking into system calls and kernel functions, this approach is more stable. Changes on kernel data structures are less frequent than code. It is also fairly straightforward to extract the data structure information from the public symbols of guest OSes.

5 PRECISE LOSSLESS DYNAMIC TAINT ANALYSIS

The primary limitation of all dynamic taint analysis implementations is the runtime performance penalty imposed upon the guest system under analysis. This penalty becomes even greater when multiple taint sources are tracked separately using unique taint labels. Tracking the propagation of multiple taint labels requires either a single heavyweight taint propagation operation that accommodates all tracked labels or multiple lightweight taint propagation operations (one for each tracked label). Neither of these approaches scale when using a large number of taint labels, imposing a limit on the number of taint labels in use simultaneously.

DECAF ameliorates this limitation by performing precise, lightweight taint *status* propagation inline with guest execution while an asynchronous, heavyweight taint propagation of multiple taint *labels* is performed in parallel to the guest execution. DECAF implements its lightweight taint propagation mostly at the TCG instruction level, so it is easily extended to support multiple CPU architectures. To achieve bit-level precision, DECAF propagates tainted bits through CPU registers, memory, and IO devices.

5.1 Taint Propagation in CPU Registers

DECAF creates TCG global variables to shadow the TCG global variables that represent general-purpose and flag CPU registers. Each shadow variable is the same size as the variable that it shadows, and each bit of the shadow variable represents the taint associated with the analogous bit in the variable. For example, the global variable `eax` for an x86 guest is shadowed by `taint_eax`, `ebx` is shadowed by `taint_ebx`, etc. When `eax` contains tainted data, `taint_eax` contains a bitmask that marks which bits of `eax` are tainted. These shadow variables emulate a set of dedicated taint-tracking registers in the guest CPU. DECAF also creates a shadow temporary variable on-the-fly to shadow each temporary variable present inside each TB. For the x86 target, we create shadow variables for the `cc_src`, `cc_dst` global variables so that taint propagates to CC flags naturally.

Currently, DECAF does not create a shadow memory for the FPU stack and the MMX stack, and we do not have special tainting rules for instructions that operate on these stacks. This is a design decision common in security applications and we leave it as a future work to investigate sound

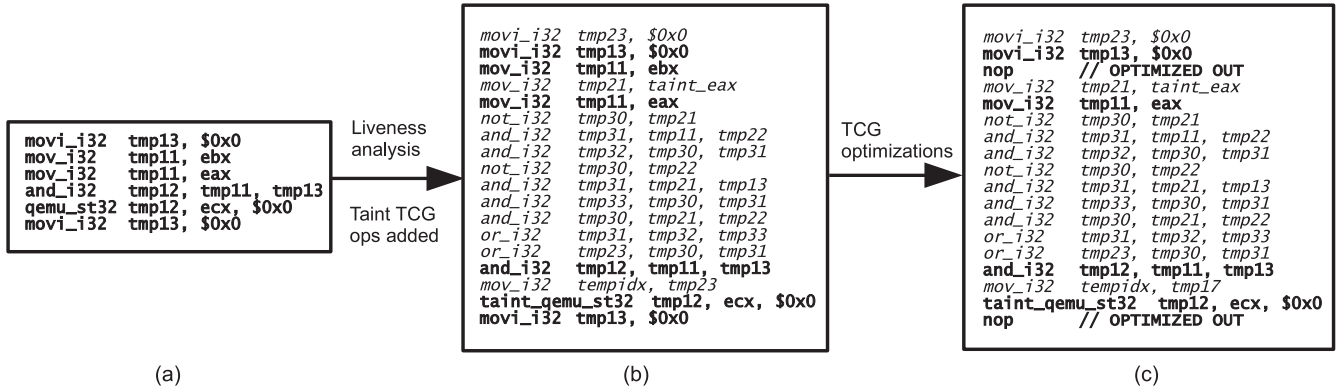


Fig. 5. Register liveness tests determine which TCG instructions in the TB (a) should be instrumented for taint propagation, and instrumentation is inserted as needed (b). TCG's optimization logic eliminates unnecessary opcodes, resulting in an optimized, instrumented TB (c).

and precise tainting rules for the floating point and MMX/SSE instructions.

Once TCG translates guest instructions into a TB containing TCG instructions, DECAF performs a translation pass on the TB to insert additional TCG instructions which implement taint propagation rules that shadow each of the original TCG instructions. For example, Fig. 5 shows that the instruction `mov_i32 tmp11, eax` is shadowed by `mov_i32 tmp21, taint_eax`. Some tainting rules are far more complex in order to be precise. For example, the add operation in Fig. 5 requires nine extra TCG instructions to precisely propagate the taint bits from two source operands to the destination. DECAF's tainting rules have been formally verified to be sound (guarantee of no under-tainting at instruction level), and most of them have also been verified to be precise (guarantee of no over-tainting). The details are presented in Section 6.

Fig. 5 illustrates this instrumentation pass. TCG translates a basic block of guest instructions into a TB of TCG instructions (a). DECAF performs its instrumentation pass on this TB by first performing a variable liveness analysis on the TCG code to determine if any TCG instruction is unnecessary or redundant. A TCG instruction that fails this analysis will be removed by TCG's optimization later, so there is no need to instrument it. Each opcode to be instrumented is compared against DECAF's list of tainting rules to determine which TCG instructions must be inserted to instrument it. The instrumentation TCG instructions are inserted prior to the original TCG instruction because some tainting rules (e.g., `and`, `or`) depend upon the values held in both the variables and shadow variables when determining taint propagation. Values held in the variables may change if the same variable is used as both the source and destination of the TCG instruction. Once this pass is complete, the TB now contains both the original and instrumentation code (b). The TCG engine performs an optimization pass on the instrumented TB and generates the final, optimized TB (c), which is then translated into the native instructions of the host and executed.

By implementing tainting rules at the IR level and with some special helper functions, DECAF is able to provide full tainting support for all integer-based x86 instructions and a few floating point and SSE instructions with simple semantics (totaling 369 opcodes-operand types and widths ignored). A

complete list of mnemonics with respect the soundness and precision guarantees can be found in Table 1.

5.2 Taint Propagation in Memory and IO Devices

The guest's physical RAM is shadowed bit-for-bit by a three-level shadow page table. While other instrumentation platforms perform byte-level precision tainting of RAM [8], [10], [24] by representing each byte of taint as a single bit, that approach requires bit masking and shifting operations to represent a 32-bit register in a 4-bit space. DECAF's bit-level precision of shadow memory ensures that taint precision is not lost as taint propagates throughout the guest.

An implementation challenge is to re-factor the existing TCG instructions that access guest memory (`qemu_ld/st`) to also access shadow memory at the same time. This is

TABLE 1
DECAF Supported x86 Instructions

AAA* AAD* AAM* AAS* ADC ADD AND ARPL BOUND BSF BSR BSWAP BTC BTR BTS BT CALLF CALL CBW CDQ CLC CLD CLI CLTS CMC CMOVB/CMOVC/CMOVNAE CMOVBE/CMOVNA CMOVL/CMOVNGE CMOVL/CMOVNG CMOVNB/CMOVAE/ CMOVNC CMOVNBE/CMOVA CMOVNL/CMOVGE CMOVNLE/ CMOVG CMOVNO CMOVNP/CMOVPO CMOVNS CMOVNZ/ CMOVNE CMOVQ CMOVPL/CMOVPE CMOVQ CMOVZ/CMOVE CMPS CMPXCHG8B CMPXCHG CMP CPUID CWDE CWD DAA* DAS* DEC DIV ENTER FWAIT/WAIT HINT_NOP HLT IDIV* IMUL* INC INS INTO INT1/ICEBP INT INVD INVLPG IN IRET JB/JNAE/JC JBE/JNA JCXZ/JECXZ JL/JNGE JLE/ JNG JMPF JMP JNB/JAE/JNC JNBE/JA JNL/JGE JNLE/JG JNO JNP/JPO JNS JNZ/JNE JO JP/JPE JS JZ/JE LAHF LAR LDS LEA* LEAVE LES LFS LGDT LGS LIDT LLDT LMSW LOCK LODS LOOPNZ/LOOPNE LOOPZ/LOOPE LOOP LSL LSS LTR MOVBE MOVSB MOVSW MOVZX MOV MUL* NEG NOP NOT OR OUTS OUT POPAD POPA POPFD POPF POP PUSHAD PUSHA PUSHFD PUSHF PUSH RCL RCR RDMSR RDPIC RDTSCP RDTSC REPNZ/REPNE REPZ/REPE REP RETF RETN ROL ROR RSM SAHF SAL/SHL SAR SBB SCAS SETB/SETNAE/ SETC SETBE/SETNA SETL/SETNGE SETLE/SETNG SETNB/SETAE/SETNC SETNBE/SETA SETNL/SETGE SETNLE/SETG SETNO SETNP/SETPO SETNS SETNZ/ SETNE SETO SETP/SETPE SETS SETZ/SETE SGT SHL/ SAL SHLD SHRD SHR SIBT SLDT SMSW STC STD STI STOS STR SUB SUB SYSENTER SYSEXIT TEST VERR VERW WBINVD WRMSR XADD XCHG XLAT/XLATB XOR
--

The tainting rules for all these instructions are sound, and most are also precise. The imprecise ones are marked with “*”.

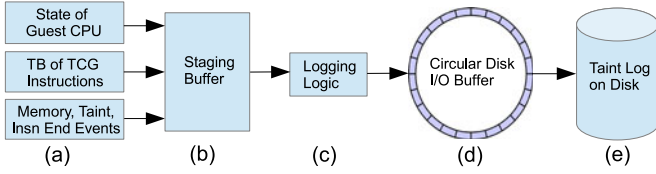


Fig. 6. All events (a) are logged into a staging buffer (b). Logging logic (c) decides which events should be recorded and places them into a circular buffer (d) that is asynchronously written to disk (e).

necessary to ensure that taint propagation occurs at the same time that memory accesses occur. The inlined SoftMMU code already uses most of the host's x86 registers for TLB lookup and parameter passing, meaning that the stack must also be used for passing taint information. This causes performance degradation and potential side effects if unexpected register spillage occurs when taint information is fetched from the stack. To counter this problem, additional shadow global variables are used specifically for copying taint information to and from the shadow page table.

Taint propagation in DECAF's virtualized devices (NE2000 NIC, IDE hard disk, PS/2 keyboard) is similar to taint propagation in memory. Each instrumented virtual device has a device-specific shadow memory, and a specific global variable passes taint data back and forth between device and RAM when programmable I/O or DMA operations occur.

5.3 Asynchronous Tainting

DECAF's lightweight taint propagation occurs inline with guest execution so that DECAF can halt execution at the exact moment that taint reaches a specific taint sink (i.e., instruction pointer, system call, virtual device). Asynchronous heavyweight taint propagation relies upon DECAF's Instruction Tracer plugin to efficiently log the taint propagation history. While the plugin is designed to log TCG instructions to record instruction traces, DECAF's flexible plugin interface enables Instruction Tracer to also record memory accesses, CPU states, and taint events. The plugin quickly logs enough information about the taint propagation for the log to be processed asynchronously offline by any custom analysis tool that executes as a separate process. Such tools can consume the taint log information as it is generated (running simultaneously with DECAF) or after DECAF's taint log has completed, performing a much more heavyweight taint analysis on the trace (i.e., reconstructing taint labels and propagation via backward slicing). The combination of synchronous lightweight and asynchronous heavyweight taint tracking guarantees that taint detection is both timely and more scalable than the inline tracking of multiple taint labels.

Fig. 6 shows the steps of the logging process. As each TB begins execution, the plugin writes an identifier for the TB and the current taint state of the CPU registers (a) to a staging buffer (b). If the TB has not been logged previously, or the TB has been flushed and retranslated since it was last logged, all TCG instructions and their arguments held in the TB are written to the staging buffer. Only the original, non-instrumented TCG instructions are written. Any memory and shadow memory accesses (both access size and both the virtual and TLB-resolved physical addresses) are

written, as are the introduction of any new taint labels. As each group of TCG instructions implementing a single guest instruction complete execution, an "instruction end" event is recorded in buffer. This is necessary because TB execution can cease early due to jumps, branches, and exceptions. There must be a record of which instructions within the TB were executed so that execution can be reconstructed. When the execution of the next TB begins, the staging buffer is examined (c). If any global shadow variable contains taint, shadow memory is accessed, or a shadow memory location is marked with a taint label, the buffer is written to a circular buffer (d) that asynchronously writes log data to disk (e). Otherwise, the staging buffer is discarded.

6 FORMAL MODEL AND DEFINITIONS

This section begins with an overview of our data-centric noninterference model used to analyze instruction level taint trackers. We also make observations on our model and how it relates to taint tracker implementations in practice. This discussion helps to motivate some of our design decisions. Finally, this section concludes with the definitions that we use for formal verification of taint propagation rules and taint analysis implementations.

The original formulation of noninterference by Goguen and Meseguer [25] was applied to a multi-level secure operating system and used a state-machine model. A more modern formulation divides the state of an arbitrary system into two parts, named "high" and "low." Consider two possible starting states of the computation that are the same in their low portions ("low-equivalent"), though the high portions may be different. If the computation satisfies noninterference, then the output states of the computation on those two inputs will also be low-equivalent. Intuitively, this definition captures a lack of information flow from high to low.

When we apply the noninterference principle to dynamic taint analysis, the tainted values correspond to high. Noninterference is a soundness property for tainting, saying intuitively that *tainted values before the computation never affect untainted values after*, or equivalently that *any value affected by a tainted value is itself tainted*. We also want precise tainting: subject to the constraint of noninterference, the amount of data tainted should be as small as possible.

The usual definition of noninterference considers the entire tainted (high) state of a system, but for reasoning about noninterference it suffices to consider the effect of changing an arbitrarily small part of the state. Stated informally, if a large change has an effect, then among the smaller changes that make it up, at least one must also have an effect. Taking advantage of this property, we narrow our analysis to consider the effect of the smallest possible change: changing a single bit from 0 to 1 or vice versa.

For the purposes of this paper, we model the state of the computation system (e.g., a CPU) as a vector of bits. We use the symbols $\wedge$, $\vee$, $\oplus$, and an overbar to represent the Boolean operations of AND, OR, XOR, and NOT either on single bits or bitvectors, equivalent to the $\&$, $|$, $\wedge$ and $\sim$ operators in C. S is the set of possible states, equal to all the bitvectors of a particular fixed size. We identify bit positions with bitvectors that have just that one bit set, and use the notation $v|_b$ for extracting a single bit b from a bitvector v .

Definition 1. Let a and b be bits in the state of a system. We say that a computation has an information flow from a to b if there are two input states s_0 and s_1 that are identical except that s_0 has $a = 0$ and s_1 has $a = 1$, and in the corresponding output states s'_0 and s'_1 , the values of b are different (one 0 and the other 1, in either order). In other words, if the computation is a function f on state vectors, and a is a bitvector with only a single position set to 1, there is a state vector $s \in S$ such that

$$f(s \vee a)|_b \neq f(s \wedge \bar{a})|_b. \quad (1)$$

From the untainted (low) perspective on a computation, tainted bits are ones whose values are unknown. Thus we can use a shorthand notation analogous to three-valued logic with three kinds of digits: 0 to represent a bit with value zero which is untainted, 1 for a bit with value one which is untainted, and ? for any tainted bit. Thus 1?0 represents a number whose second bit is tainted; in effect, the value from the high perspective might be either 4 (binary 100) or 6 (binary 110).

6.1 Taint Propagation Rules in Practice

We make three important observations about the data-centric noninterference model. First, the model is defined using information flows between bits. Thus it directly describes systems in which taint is labeled per bit. Not all implementations take this approach, but the model extends naturally to coarser-grained taint. For example, there is information flow from byte x to byte y as long as there is information flow from any bit of x to any bit of y . Results from a coarse-grained analysis are inherently limited in their precision, but for any granularity, we can try to achieve the most precise results expressible at that granularity. With DECAF, we are interested in exploring the maximum possible precision, so we focus on bit-level tainting.

Second, we observe that the precision of taint results also depends on the granularity of the computation analyzed. The reason is that the taint status of bits does not include information about how some bits might be correlated with others. For instance, suppose we take a single tainted bit ? (representing either 0 or 1) and multiply it by an untainted 3 (binary 11). The result must be either 0 (00) or 3 (11); thus both the low bits should be tainted, represented as ?? . If we know where the value came from, we know that the first and second bit positions must have the same value, but this information is missing in the tainted-bit representation, which could equally well describe a 1 (01) or 2 (10). This inherent imprecision of the representation leads, in turn, to imprecision in later results. For instance if we take the tainted bit value ?? and multiply it by 3 again (i.e., $? \times 3 \times 3$), the result is ????, since there is information flow to each of the four bits of the result. On the other hand, if instead of multiplying it by 3 twice (as two separate operations), we had started with the tainted bit ? and multiplied it by 9 in one operation, the result becomes the more precise ?00?.

This is a general phenomenon: expressing a larger computation in terms of smaller ones and applying sound taint analysis to each operation separately will always give sound final results. However, applying precise taint analysis to each operation separately will often not give as precise of a result as analyzing the entire computation at once.

At the binary level, there are two common choices for taint analysis: we can either perform the analysis and update the taint labels after each instruction, or we can translate each instruction into a sequence of operations in a simpler intermediate representation, and analyze the taint effects of each IR operation separately. Though this IR-level approach has other advantages, it can come at a cost to precision for the reason described in the previous paragraph. As an instruction-level example, consider an instruction (such as the BIC instruction on ARM) which computes the bitwise-AND of one register and the bitwise negation of another: $z = x \wedge \bar{y}$. If the two inputs are the same register, this has the effect of clearing the output register, so if this instance of this instruction is analyzed as a unit, the output should be completely untainted. On the other hand, an IR-level taint analysis that treated the AND and NOT as separate operations would be unable to tell that one operand of the AND was the negation of the other, so the result would still be tainted. Our formal verification can reveal these kinds of imprecision.

A final remark is that, as specified so far, the model does not place any further restrictions on the choice of the input state s ; the specific selection comes from the context in which we are verifying a taint analysis. To analyze the taint propagation in a particular situation, we can specify a concrete value for s . For instance, we can use a program state encountered during testing. On the other hand, in constructing rules for taint propagation, we would like them to work correctly in all situations, so we look for taint rules that will soundly and precisely capture information flow for any choice of s . In short, s is a free variable when constructing rules and s is concretized when verifying rules.

6.2 Verifying Taint Propagation Rules

Taint propagation rules have usually been defined based on domain expertise and then reasoned about manually, or simply left unverified due to the difficulties of manual verification. For example, Memcheck [26] has many special case rules, but according to its project suggestions webpage, formal verification of the rules is still needed [27]. The concepts for formal verification of tainting rules are introduced in this section.

The most obvious representation for bit-level taint, used by Memcheck, is to maintain taint bits parallel to data bits with the same structure: for instance, the taint information for a 32-bit data word is represented by another 32-bit word, with the first bit of the taint word reflecting the taint status of the first bit of the data word, etc. DECAF's implementation of shadow memory also uses a bit-for-bit mapping. We adopt the convention that a taint bit value of 1 indicates that the corresponding data bit is tainted, while 0 indicates untainted. Memcheck uses the opposite convention in its implementation (for what are referred to as validity or "V" bits), but because of the duality of Boolean algebra, the choice makes little difference.

We will use the suffix $_{\mathcal{t}}$ for variables holding taint; for instance $S_{\mathcal{t}} = S$ is the set of all possible taint states. The taint propagation rule for a given operation is a function that takes as inputs the data state before the operation and the taint state before the operation, and yields the taint state after the operation: $\text{rule}_{\text{op}} : S \times S_{\mathcal{t}} \rightarrow S_{\mathcal{t}}$. Our definition of a sound and precise rule is that the taint bit for an output

position b should be set if (soundness) and only if (precision) there is an input bit position a for which there is information flow from a to b and a is tainted.

An equivalent perspective on the soundness of a rule, analogous to noninterference, is that for each bit position b that is untainted after an operation, it should be the case that for any choice of values for the tainted input bits, the value of that untainted output bit is constant. If this condition fails, and there is an output bit that is affected by the tainted input but is not itself tainted, we say that the rule suffers from a false negative error. As a formula, let $y.t$ be the output taint after applying the rule for the operation f to the input data state x and the input taint $x.t$. We make the following definition.

Definition 2. A rule $y.t = \text{rule}_f(x, x.t)$ applied to an operation $y = f(x)$ has a false negative error if

$$\exists b, x_1, x_2 : (y.t|_b = 0) \wedge ((x_1 \wedge \overline{x.t}) = (x_2 \wedge \overline{x.t})) \wedge (f(x_1)|_b \neq f(x_2)|_b)$$

for some bit position b . Equivalently we can also compare all the untainted output positions at once

$$\exists x_1, x_2 : ((x_1 \wedge \overline{x.t}) = (x_2 \wedge \overline{x.t})) \wedge ((f(x_1) \wedge \overline{y.t}) \neq (f(x_2) \wedge \overline{y.t})). \quad (2)$$

Conversely, a rule has a false positive error if there is a bit position which is tainted, but does not in fact depend on the tainted input:

Definition 3. A rule $y.t = \text{rule}_f(x, x.t)$ applied to an operation $y = f(x)$ has a false positive error if

$$\exists b : (y.t|_b = 1) \wedge \forall x_1, x_2 : ((x_1 \wedge \overline{x.t}) = (x_2 \wedge \overline{x.t})) \Rightarrow (f(x_1)|_b = f(x_2)|_b). \quad (3)$$

Observe that the input state variables x and $x.t$ are free in Equations (2) and (3). When checking the taint propagation in a trace, we instantiate them with values taken from an execution. When checking the correctness of a rule in the abstract, we quantify over all possible values for x and $x.t$: a rule is sound if there is no value of x and $x.t$ for which Equation (2) holds, and precise if there is no value of x and $x.t$ for which Equation (3) holds.

6.3 Constructing Tainting Rules

In the previous section, we presented a formal model for taint analysis based on noninterference, and defined soundness and precision based on information flow. In security applications, unsoundness can lead to missed attacks (a result we consider worse than false alarms), so we choose to first construct a set of rules that are guaranteed to be sound, and then refine them to maximize precision.

This section focuses on the key concepts in constructing precise tainting rules. The reader can refer to our technical report [28] for a more detailed treatment of topic, including examples of how these were actually applied for DECAF. Constructing tainting rules is separated into three key steps. First, sound tainting rules are constructed by identifying all bit-wise information flows in operations. Second, SMT solvers are used to verify that the rules are indeed sound. Third, the sound rules are improved upon to create precise rules;

these precise rules are formally verified as well. We draw our examples from the x86 instruction set, but the techniques and most of the specific rules are applicable to other architectures, since the same basic operations (such as addition and bit shifts) are provided by all CPUs.

6.3.1 Constructing Sound Rules

Recall that a rule is sound if every information flow from a tainted input bit to an output bit is noted by making the output bit tainted. Thus, to construct a sound rule, we first identify all possible information flows within an instruction and then summarize these flows with a rule. Since definition 1 is a satisfiability problem, we use satisfiability-module-theories (SMT) solvers to identify the information flows. To do this, we first model the behavior of each instruction of interest using the bitvector operations of SMT solvers. Then, we submit queries to SMT solvers to identify all information flows. For DECAF, all of the instructions are modeled in SMT-LIB Version 2 [29] (“SMT2” for short) to maintain compability with a wide range of solvers.

Stage 1: Behavioral Definitions. There are two general ways to define the behavior of instructions: manual and automatic. Godefroid and Taly [30] presented algorithms to automatically generate the behavioral specifications of common x86 instructions. The key intuition behind their approach is that many instructions follow specific behavioral “templates” (e.g., an *addition* template will cover the `add`, `sub`, `inc` and `dec` instructions). Thus, their algorithms use a small number of manually defined templates to automatically specify the behaviors of a large number of instructions. While an automated approach is available, we chose the manual approach since our experience with the x86 instruction set and BAP [31] allowed us to manually define the behaviors quicker than reimplementing Godefroid and Taly’s algorithms. Additionally, templates for special instructions such as `cmpxchg` were not readily available. This meant that we had to define them manually anyways.

Since the correctness of the behavioral definitions is paramount, we relied on both BAP and the developer’s manuals to help define the models. Please note that if any errors exist in the behavioral definitions at this point, they will be revealed when Per-Trace Verification (described in Section 7.2) fails as well. To manually define the behavior of the x86 instructions, we first divided the instruction set into four categories: *data transfer*, *control transfer*, *arithmetic and logic*, and *special*. Data and control transfer instructions have simple semantics with obvious bitwise information flow relationships and do not warrant further analysis. The arithmetic and logic category includes instructions that are likely to be supported in any general-purpose architecture. We focus on these instructions for wider applicability. The rest of the instructions fall into the *special* category. The `cmpxchg` is a prime example of a special instruction since it has an unusual information-flow pattern.

For all of the instructions of interest, we wrote assembly code to exercise different aspects of their behavior, linked them into an executable, and then lifted the executable into BAP’s internal IR (BIL). This resulted in a collection of BIL that summarizes the instruction. We then extrapolated a single SMT2 behavioral representation

TABLE 2
 Flow Type Results for x86 Instructions

Instruction	Inputs	Outputs	# Cases	Runtime	Flow Type	Droid Scope [24]	libdft [32]	Minemu [17]	TEMU [33]	Memcheck [26]	DECAF
<i>adc dst, src</i>	dst,src,cf	dst,src,zf,of,sf,af,cf,pf	4,550	1m19s	U	A	I	A	S	U	S
<i>add dst, src</i>	dst,src	dst,src,zf,of,sf,af,cf,pf	4,480	1m13s	U	A	I	A	A	S	S
<i>and dst, src</i>	dst,src	dst,src,zf,sf,pf	4,288	1m05s	I	A	I	A	I	S	S
<i>dec dst</i>	dst	dst,zf,of,sf,af,pf	1,184	20s	U	A	I	A	A	U	S
<i>div rm</i>	edx,eax,rm	edx,eax,rm	9,216	95m48s	D	A	I	N	A	A	D
<i>idiv rm</i>	edx,eax,rm	edx,eax,rm	9,216	307m04	A	A	I	N	A	A	A
<i>imul1 rm</i>	eax,rm	edx,eax,rm,of,cf	6,272	289m51s	U	A	I	N	A	U	U
<i>imul2 dst, rm</i>	dst,rm	dst,rm,of,cf	4,224	52m37s	U	A	I	N	A	U	U
<i>imul3 dst, rm, imm</i>	rm,imm	dst,rm,imm,of,cf	6,272	53m56s	U	A	I	N	A	U	U
<i>inc dst</i>	dst	dst,zf,of,sf,af,pf	1,184	19s	U	A	I	A	A	U	S
<i>mul rm</i>	eax,rm	edx,eax,rm,of,cf	6,272	16m02s	U	A	I	N	A	U	U
<i>not dst</i>	dst	dst	1,024	15s	I	A	I	A	I	I	I
<i>or dst, src</i>	dst,src	dst,src,zf,sf,pf	4,288	1m05s	I	A	I	A	I	S	S
<i>rcl dst, imm8</i>	dst,imm8,cf	dst,imm8,of,cf	1,722	42s	A	A	N	A	A	A	S
<i>rcr dst, imm8</i>	dst,imm8,cf	dst,imm8,of,cf	1,722	42s	A	A	N	A	A	A	S
<i>rol dst, imm8</i>	dst,imm8	dst,imm8,of,cf	1,680	41s	A	A	N	A	A	S	S
<i>ror dst, imm8</i>	dst,imm8	dst,imm8,of,cf	1,680	41s	A	A	N	A	A	S	S
<i>sal dst, imm8</i>	dst,imm8	dst,imm8,zf,of,sf,af,cf,pf	1,840	35s	U	A	N	A	S	S	S
<i>sar dst, imm8</i>	dst,imm8	dst,imm8,zf,of,sf,af,cf,pf	1,840	34s	D	A	N	A	S	S	S
<i>sbb dst, src</i>	dst,src,cf	dst,src,zf,of,sf,af,cf,pf	4,550	1m21s	U	A	I*	A*	A	A	S
<i>shr dst, imm8</i>	dst,imm8	dst,imm8,zf,of,sf,af,cf,pf	1,840	35s	D	A	N	A	S	S	S
<i>sub dst, src</i>	dst,src	dst,src,zf,of,sf,af,cf,pf	4,480	1m17s	U	A	I*	A*	A*	S	S
<i>xor dst, src</i>	dst,src	dst,src,zf,sf,pf	4,288	1m05s	I	A	I*	A*	A*	I	I
<i>bsf dst, src</i>	src	dst,src,zf	2,080	31s	A	N	I	N	A	A	S
<i>bsr dst, src</i>	src	dst,src,zf	2,080	31s	S	N	I	N	A	A	S
<i>cmpxchg rm, r</i>	eax,rm,r	eax,rm,r,zf,of,sf,af,cf,pf	9,792	2m39s	S	N	E	N	E	E	S
TOTAL			102,064	13h52m48s							

Flow Types: (U)p, (D)own, (I)n-place, (A)ll-around, (S)pecial, (N)ot-Supported, (S)pecial, (E)ax is tainted in *cmpxchg*, *—Zeroing Idiom, **Boldface**—Generated Policy is more precise.

from the instruction’s BIL instances cross-checked against the processor documentation.

In total, we analyzed over 150 different arithmetic and logic instructions. After some initial tests, we found that the precise mnemonics and operand choices (e.g., *add r/m8, r8* versus *add r8, r/m8* versus *add r16/32, r/m16/32*), did not affect the information flow patterns. Thus, we decided to simply focus on generic 32-bit register instruction formats (e.g., *add dst, src*). Our 26-instruction test set is outlined in Table 2. Please note that, similar to Godefroid and Taly’s intuition on templates, while we have only focused on these 26 instructions, our design for DECAF enables us to use the rules for these 26 instructions to support most of the x86 instructions.

Stage 2: From Information Flow to Sound Rules. The goal of this stage is to take the SMT2 files from stage 1 and identify all possible information flows. For each file, we iterate through all possible pairs of input and output bits and query Z3 [34] for the satisfiability of the condition in Definition 1. A “sat” result means that there is information flow and an “unsat” result means there is none. Fig. 7 shows two (simplified) example queries. The resulting statistics for all the instructions are summarized in the first five columns of Table 2. The instructions are presented in the first column; the input operands, both implicit and explicit, in the second; output operands, both implicit and explicit, in the third; the total number of input-bit to output-bit combinations in column four; and the time it took for Z3 to process the queries

is shown in column five. We used a new instance of Z3 for each test case and thus the timing results include process creation overhead.

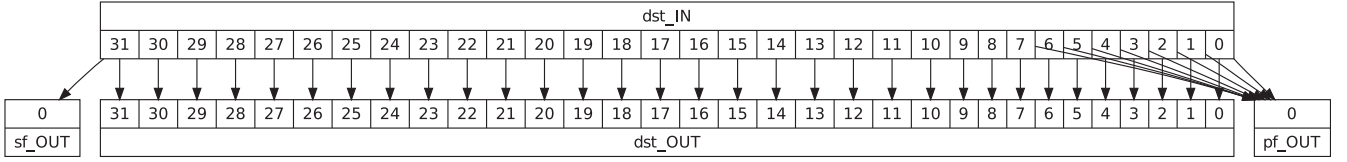
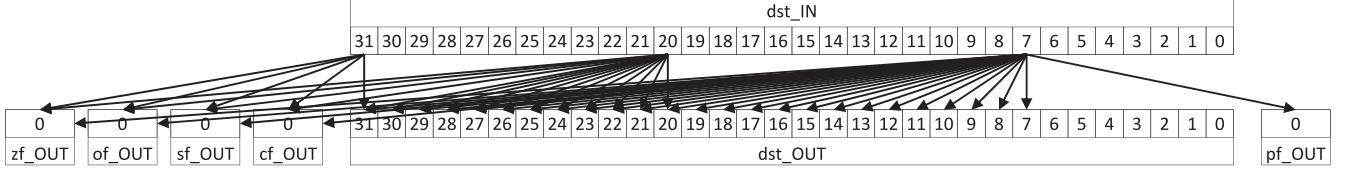
As expected, logical operations return results extremely quickly whereas signed multiply and divide takes the most time. Overall, it took less than 14 hours on an Intel Core-i7

```
(declare-fun x1 () (_ BitVec 32))
(declare-fun x2 () (_ BitVec 32))
(declare-fun a () (_ BitVec 32))
(assert (= a #x00000001))
(declare-fun b () (_ BitVec 32))
(assert (= b #x00000010))

;; For NOT:
(assert
  (not
    (= (bvand b (bvnot (bvor x1 a)))
      (bvand b (bvnot (bvand x1 (bvnot a)))))))

;; For ADD:
(assert
  (not
    (= (bvand b (bvadd (bvor x1 a) x2))
      (bvand b (bvadd (bvand x1 (bvnot b))
                      x2)))))
```

Fig. 7. Example SMT queries checking for information flow (Equation (1)) from the low bit *a* of an input to the 4th bit *b* of an operation output. The first query, for *not*, is unsatisfiable, indicating no flow. The second query, for *add*, is satisfiable, for instance by *x1* = 0 and *x2* = 0x1: there is flow.

Fig. 8. Information flow of *dst* in *or* instruction.Fig. 9. Information flow of bits 7, 20 and 31 of *dst* in *sbb* instruction.

860 to automatically identify all information flow relationships for the arithmetic and logical instructions.

The Rules. Once all of the possible information flows were revealed, we then summarized the flows into simple rule types. The sixth column of Table 2 indicates the general flow type for each instruction. There are four distinct information flow patterns between the source and destination operands. We reserve a fifth type, *special*, for more complex cases. The four basic flow types serve as four different sound rules that we will refine later on. Note that we did not find any patterns of interest for the flags.

The four rules are: 1. *In-place*: Information can only flow from bit *i* of the source to bit *i* of the destination (as shown in Fig. 8). 2. *Up*: Information can only flow from bit *i* of the source to bits *j* of the destination where $j \geq i$. Fig. 9 depicts this behavior, showing the combination of the information flow graphs for bits 7, 20, and 31. It is evident from the figure that information only flows from bit 7 of the source operand to bit 7 and higher of the destination. The same applies to bits 20 and 31, where bit 31 of the source only flows to bit 31 of the destination. 3. *Down*: Dual to up, information can only flow from bit *i* of the source to bits *j* of the destination where $j \leq i$. 4. *All-around*: Information can flow from bit *i* of the source to any bit of the destination.

We stress that there are times when a single instruction requires multiple tainting rules. Table 2 is not an exhaustive list. The divide instructions are good examples of this. In the divide operation, *edx:eax* is divided by *rm*, the quotient placed into *eax* and remainder into *edx*. Intuitively, division is similar to shift right and thus the flow type for *edx:eax* to *eax* should be *down*. On the other hand, the flow type for *edx:eax* to *edx* is *all-around* since nothing definitive can be said about the relationship between the divisor and the remainder without concrete value analysis.

Special Instructions. Implicit information flows (those due to control dependencies) are a known source of imprecision in taint analysis; they can even occur within a single

instruction, making tainting rule definition difficult. The *bsf*, *bsr*, and *cmpxchg* instructions exhibit such behavior and are thus considered special.

We use the *cmpxchg* (Fig. 10) x86 instruction to illustrate a potential pitfall. Applying Definition 1 to the instruction shows that there is no information flow from *eax* to *eax* because the output value of *eax* is fully dependent on the input value of *rm32*. On the other hand, if information flow was analyzed line-by-line using the technique proposed by Ferrante et al. [35] (both Dytan [14] and DTA++ [36] use this technique), *eax* will be tainted if *eax* was tainted before the instruction. This is because *eax* was unchanged in the equals branch (line 3) and thus retains its taint. The case for simple control flow dependencies is even worse. Since *eax* is used in the comparison on line 2 and also as an l-value on line 4, it will remain tainted in the not-equals branch. The false positive arises from the fact that the above mentioned techniques analyzed the information flows line-by-line—this is what IR level tainting does—, thus knowledge of the logic in the other branch is not taken into consideration. Overall, striking a balance between handling all possible special instructions and only handling a smaller subset of instructions that can be used to emulate the rest is a design decision. In DECAF, we chose a middle ground that leaves some special cases unhandled, but uses per-trace verification to minimize errors.

6.3.2 Constructing Precise Rules

The previous section focused on the construction of sound rules. We arrived at four basic rules that are sound by construction. However, we ended the section with a discussion of *cmpxchg*, which motivated the need for formal verification of tainting rules. Tainting rule verification is accomplished in two steps: the operation and the tainting rules under test are formally specified, and then solvers are used to determine whether Equations (2) and (3) are satisfiable. The formal specification step is straightforward using the models from Stage 1 and will not be discussed further. When we verified the sound rules from the previous section, we found that while all of the rules were sound (as expected), many of them were not precise. In order to construct precise rules, we sought inspiration from Memcheck, since it has many specially-defined rules. What we found was that many of Memcheck’s rules were in fact precise, and thus we simply needed to improve upon those that

```

1. cmpxchg (rm32, r32) {
2.   if (eax == rm32) then
3.     rm32 = r32;
4.   else eax = rm32;
5. }

```

Fig. 10. Pseudocode for *cmpxchg* (flags are omitted).

TABLE 3
Precise Rules and Verification Results: Length
of Operands Verified (in Bits)

Operation	Sound	Precise		Operation	Sound	Precise	
		Z3	MONA			Z3	MONA
add	256	16	✓	adc	256	16	✓
and	256	256	✓				
cmpEq	256	256					
or	256	2	✓				
rol	256	16*		rcl	256	16*	
ror	256	16*		rcr	256	16*	
sal/shl	256	16*					
sar	256	16*					
shr	256	16*					
sub	256	4	✓	sbb	256	4	✓
				bsf	32		16 ^z
				bsr	32		16 ^z

✓Verified for all lengths. *Shift amount is untainted. ^zNon-zero operand for *bsf*, *bsr*.

weren't. In total, we added six new precise rules for *adc*, *sbb*, *rcr*, *rcl*, *bsf*, and *bsr*, summarized in Table 4. We have included example SMT2 code for a 2-bit and verification example in Appendix A, which can be found on the Computer Society Digital Library at <http://doi.ieeecomputersociety.org/10.1109/TSE.2016.2589242>, to provide a more detailed example of the process.

The verification results of all specially defined rules are summarized in Table 3. We placed the Memcheck rules on the left and our rules on the right side of the Memcheck rules if the rules are similar. There are four columns: the operation, Z3 result for soundness, the Z3 result for precision and finally, if the Z3 result was inconclusive (i.e., Z3 did not return a result after 24 hours of processing), the MONA [37] result of whether the rule is precise, and the corresponding rule that we verified. Note that we chose MONA as a complementary decision procedure to Z3 since it deals gracefully with alternating quantifiers, which Z3 does not. On the other hand, MONA is less expressive, making it difficult to use MONA for all cases.

As the results show, all of the special rules defined in Memcheck are sound for operands up to 256 bits³. Additionally, the special rules for *and* and *cmpEq* are also precise up to 256 bits. In most cases, Z3 times out for operands beyond 16 bits in length. We hypothesize that the size of the state space to explore is the culprit since smaller bit lengths returned quickly. MONA was able to verify precision of the *add*, *adc*, *or*, *sub* and *sbb* rules.

All of the shift rules were shown to be imprecise. This is because the shift amount can be tainted, which causes all bits of the output to be marked as tainted. Subsequently, we asserted that the shift amount is not tainted, and re-verified the rules. They were shown to be precise for up to 16 bit operands using Z3.

7 EVALUATION

We evaluated the performance overhead of DECAF under different configurations (such as VMI and tainting), verified

the correctness of DECAF's tainting implementation using *per-trace verification*, and evaluated DECAF's analysis capabilities using three plugins (API Tracer, Keylogger Detector, and Instruction Tracer). The source code for these plugins are available for download from DECAF's project page [18].

The hardware used for all evaluations is a 32-core 2.0 GHz Intel Xeon ES-2650 CPU server with 128 GB of RAM. The server uses Ubuntu 12.04 Linux (3.2.0 kernel) as its OS. DECAF was executed on this server using an ARM Debian 6.0 Linux (2.6.32 kernel) VM image and three x86 guest VM images: Windows 7, Windows XP SP3 and Ubuntu 12.04 Linux (3.2.0 kernel). 4 GB of RAM was allocated to each of the x86 VMs, and 128 MB of RAM was allocated to the ARM VM. The priority of DECAF was nice'd to -20 to ensure that it would be minimally influenced by other processes executing on the benchmark hardware.

7.1 SPEC CPU2006 Benchmarks

We evaluated DECAF's instrumentation performance impact using the CINT2006 integer component of the SPEC CPU2006 benchmark suite.⁴ We chose the CINT2006 tests because the tainting instrumentation is applied to the TCG instructions, which all implement RISC-like integer operations. Floating point operations are implemented as a set of guest architecture-specific helper functions. Performance of ARM VMs under DECAF cannot be measured using the benchmark suite due to the memory requirements of the tests. The majority of the tests exceed the RAM allocated to the VM.⁵ and will measure the performance of the memory paging to disk, rather than the instrumented operations of interest. While a direct comparison of TEMU and DECAF performance using these benchmarks would be informative, this is infeasible because TEMU is too slow to correctly execute the tests. We attempted to execute the benchmark suite under TEMU, but the first benchmark test of the suite (400.perlbench) was allowed to run for over a day before its execution was terminated.

Baseline DECAF, without any instrumentation enabled, experiences an average of 15.20 percent overhead over the execution performance of a similarly-configured QEMU. DECAF updates EIP (x86) and R15 (ARM) after every guest instruction to ensure accurate analysis, while QEMU updates these registers at the end of each TB. DECAF must also maintain its plugin infrastructure by continually watching for the registration of new plugin callbacks.

The VMI overhead measurements in Fig. 11 show the difference in performance between running DECAF in a baseline configuration with all features disabled and a configuration with only VMI enabled. Average overhead is 12.07 percent for Windows 7 and 14.48 percent for Linux. The negative overhead result for the Linux 400.perlbench test can be attributed to the short execution time of the test and the general variability in execution times within an emulated VM environment. The result of 429.mcf has considerably higher VMI overhead than the other tests with 54.36 percent for Windows 7 and 55.23 percent for Linux.

4. 462.libquantum was omitted from the test suite due to Visual Studio's Visual C++ not supporting some C++ features used by the test.

5. The "versatilepb" platform QEMU uses to emulate ARM VMs has a 256 MB RAM limitation.

3. We chose 256 bits as the maximum length to test, since we are unaware of any architectures with operands greater than 256 bits.

TABLE 4
New Precise Bit-Level Taint Rules: `rcr` and `bsr` Are Similar to `rc1` and `bsf` Respectively, and So Omitted

Operation	Rule (C-like pseudocode)
<code>adc</code>	$x1_min = x1 \& \sim t1; x2_min = x2 \& \sim t2; cf_min = cf \& \sim tcf;$ $x1_max = x1 t1; x2_max = x2 t2; cf_max = cf tcf;$ $t1 t2 ((x1_min + x2_min + cf_min) \wedge (x1_max + x2_max + cf_max))$
<code>sbb</code>	$t1 t2 ((x1_min - (x2_min + cf_min)) \wedge (x1_max - (x2_max + cf_max)))$
<code>rc1</code>	$pcast(v) \{ v == 0 ? 0 : -1 /* all ones */ \}$ $pcast(t2) rc1(t1, x2, tcf)$
<code>bsf</code>	$xc = x1_max \& \sim ((x1_min < 1) -(x1_min < 1));$ $((xc \& 0x5555) \& (xc \& 0xaaaa) ? 1 : 0) $ $((xc \& 0x3333) \& (xc \& 0xcccc) ? 2 : 0) $ $((xc \& 0x0f0f) \& (xc \& 0xf0f0) ? 4 : 0) $ $((xc \& 0x00ff) \& (xc \& 0xff00) ? 8 : 0);$

The `bsf` rule is shown for a 16-bit value which must be non-zero, and the rule for `rc1` is precise only when the rotate amount is untainted. $x1$, $x2$, and cf (carry flag) are the operands while $t1$, $t2$, and tcf are the respective shadow taints.

This test incurs almost twice as many TLB misses as the next closest test (471.omnetpp). VMI callbacks are triggered when TLB misses occur, explaining the larger amount of observed overhead.

Furthermore, Tables 5 and 6 present the boot time overhead and source code distribution between architecture dependent and independent components. DECAF and VMI impose a combined overhead under 25 percent on x86 and 8.72 percent on ARM. Also, from Table 6 we can see that most of the plugin code is architecture independent. API Tracer includes OS-specific code to interpret some OS-specific data structures, but, the core part of API Tracer contains no OS-specific code.

The inline taint propagation measurements in Fig. 11 show the difference between running DECAF in a baseline configuration with all features disabled and with inline tainting enabled for the Windows 7 VM. The average overhead is 605.07 percent, ranging from 285.32 percent (429.mcf) to 815.77 percent (458.sjeng). Taint propagation overhead is directly related to the number of TCG instructions being executed, so it is highest for CPU-bound tests. Because DECAF's inline tainting executes multiple taint propagation TCG instructions for each TCG instruction that executes, an average slowdown of six-times is justified.

We used the internal QEMU profiler (the `info jit` QEMU monitor command) to obtain the translation block (TB) statistics. For the QEMU baseline, we found that the average TB contains 45.3 IR instructions with the largest TB having 464 instructions. An average of 29.3 temporary registers were used by the TBs, with a maximum of 68 temporary registers used. On the other hand, DECAF TBs have an average of 86.7 IR instructions with the largest TB

containing 520 instructions. On average, 74.0 registers were used with a maximum of 358 temporary registers.

7.2 Per-Trace Verification of DECAF's Tainting

We use *per-trace verification* as a technique to verify the correctness of a taint analysis system's implementation. A high level overview of the process is depicted in Fig. 12.

In per-trace verification, the taint analysis system under test (e.g., DECAF) executes a program and generates a tainted trace. The trace is a log of all instructions executed, along with additional metadata. Each log entry contains the instruction executed, the input operand values, the output operand values, and the corresponding taint label assignments. (A sample log entry is shown in Fig. 13).

For each entry in the instruction trace, an *oracle* is used to determine whether the resulting taint matches the noninterference model. The oracle consists of three main components. An *IL translator* is used to translate the operation (and in the example) into a bitvector formula. A *query generator* then takes the translated formula, the concrete values from the trace entry, and the input taint assignments and generates a query to determine the correct output taint labels. This query is subsequently sent to an SMT solver and the results compared to the output taint as recorded in the trace entry. If they agree, then the implementation is correct for this particular operation and machine state. If they disagree, either the rule is imprecise or there is an implementation bug.

Per-trace verification has a number of advantages. First, the traces can be generated and verified independently and thus processed in parallel. Second, the problem of verifying traces one instruction instance at a time is more tractable: using concrete values reduces the state space to explore.

TABLE 5
Execution Overhead for DECAF and DECAF with VMI on Different Architecture/OSs without Tainting

Setup	XUbuntu	WinXP SP 3	Debian Squeeze (ARM)
DECAF with VMI	3 m 25.9 s	1 m 4.36 s	2 m 50.16 s
QEMU 1.0.1	2 m 45.85 s	0 m 52.79 s	2 m 36.52 s
DECAF + VMI	24.14	21.91	8.72
Overhead %			

TABLE 6
Code Breakup of DECAF, VMI and Different Plugins

	OS/Arch independent (LOC)	OS Specific (LOC)	Total (LOC)
DECAF	18,470	1,350	19,820
Insn Tracer	3,770	90	3,860
API Tracer	840	880	1,720
Key Logger	120	0	120

The code introduced by DECAF is beyond QEMU, which by itself has over 500 K LOC.

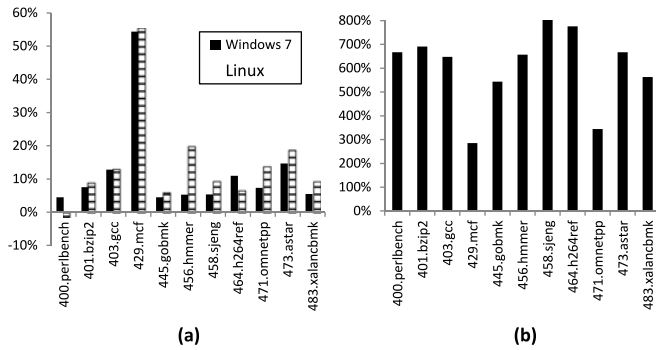


Fig. 11. CINT2006 benchmarks that measure overhead for VMI (a) and inline taint propagation (b).

Third, the oracle can also be used as a taint analysis system itself. For example, a taint analysis system might use sound but imprecise tainting rules to improve runtime performance and then use the oracle to reprocess the trace offline and remove any false positives.

The major limitation of per-trace verification is coverage. Per-trace verification will not be complete unless the traces used to verify the system cover all possible system states (i.e., all possible combinations of operations, operand values and taint values). To maximize coverage, we use a collection of over 600,000 test programs from the PokeEMU [38] project. These test programs were automatically generated by exploring all of the different instruction decode and execution paths of the Bochs x86 emulator. They provide full path coverage of more than 800 protected-mode x86 instructions, and so our per-trace verification results inherit this same extensive coverage.

In order to verify DECAF, we first implemented an instruction tracer to generate the tainted trace. We implemented the oracle using BAP as the IL translator and STP [39] as the SMT solver (Z3 works as well). Specifically we express the bitvector formula and queries in the BAP IL, allowing us to use BAP’s existing interface to STP (or Z3).

The correctness of DECAF’s rule implementations was verified using the 600,000+ PokeEMU test cases. Each test case was executed using DECAF, and all instructions executed were logged into a tainted trace, one per test case. Due to the sheer number of test cases, we did not exhaustively try all possible taint assignments to the program state. Instead, we assigned random taint values to the program state at the beginning of execution and allowed it to be propagated through the program.

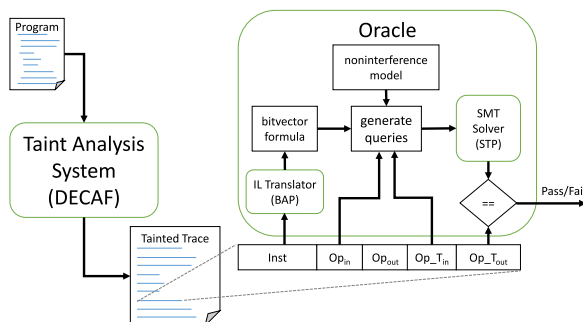


Fig. 12. Per-trace verification overview.

```
/* ebx = eax & ebx */
Inst: and %eax, %ebx
Inputs:  eax = 0x84be2329, ebx = 0xaed66ce1
Outputs: ebx = 0x84962021
Input Taints:  eax_t = 0x7369C667, ebx_t = 0xec4aff51
Output Taints: ebx_t = 0xe44ae761
/* Expected Output Taints: ebx_t = 0xe64ae761 */
```

Fig. 13. Trace entry for and bug.

Each trace was then passed through the oracle to determine whether there were any differences in the output taints. If the verification failed, we manually reviewed the offending instruction in an attempt to track down the source of the failure. If a bug was found, we patched it and then re-ran the offending test case to ensure that the bug was patched. We also re-ran similar test cases to ensure that a new bug was not introduced. In total, it took over 16 days to complete the verification task by running 80 verification instances in parallel. Each trace took approximately 3 minutes to complete. This does not include the extra time needed to address the few bugs that were discovered.

This method of verification uncovered two incorrectly implemented tainting rules in DECAF (and and add). Both errors were due to the same implementation mistake. A text version of the offending trace entry is shown in Fig. 13. The figure shows the concrete values of the operands, as well as the input and output taints. According to DECAF, the output taint was 0xe44ae761, which failed verification because the expected taint was 0xe64ae761. Notice that bit 25 is 0, but should actually be 1.

As it turns out, this error was due to the way we inserted the extra TCG IR to propagate taint in DECAF. In the code for adding the propagation code for and, we incorrectly placed the propagation code *after* the original and operation. As a result, instead of using the concrete value of 0xaed66CE1 for ebx to calculate the taint, we used the *result* of ebx (0x84962021). In fact, this bug was pervasive in our implementation, and we didn’t understand it until we discovered that the add implementation had the same problem. In general, this bug only surfaces if the destination operand is also a source operand, and the value written to the destination happens to affect the final taint calculation, meaning it depends on both the concrete values as well as the taint assignments. This discovery led to our insertion of all IRs that implement taint propagation instrumentation for an IR immediately *prior* to the IR that they instrument.

7.3 API Tracer

The API Tracer plugin leverages the VMI and function hooking features of DECAF to capture API-level traces of the user- and kernel-mode execution of a program.

At its core, API Tracer is a minimal and stand-alone cross-platform component, comprised of 340 lines of C code, that retrieves function-level execution traces of programs on any platform/OS supported by DECAF. Furthermore, we implemented a custom configuration parser, comprised of 500 lines of C code, and a Windows-specific extension component, comprised of 880 lines of C code, to decipher the higher-level OS-specific semantics. For example, in Windows the kernel32.dll::CreateProcess() API call contains newly created process information and the creation flag

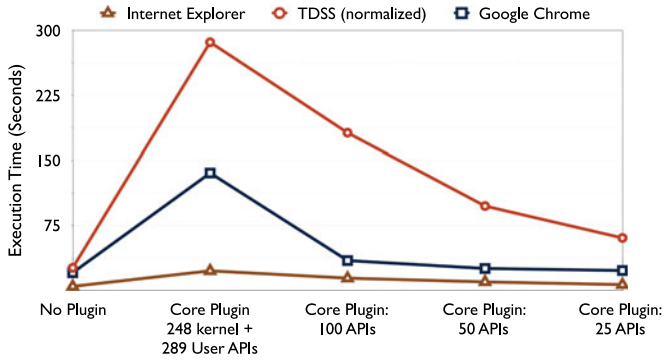


Fig. 14. Evaluation of API Tracer plugin.

parameters required to extend analysis into child processes. The OS-specific component interprets such information and acts accordingly. Unlike static analysis tools that are unable to analyze dynamically generated code, and user-space dynamic analysis tools (such as Pin [1]) that are unable to analyze activity in the kernel, API Tracer keep track of any kernel modules loaded by a user program and traces such modules automatically. It also monitors the memory allocation and deallocation of a program to identify and trace any unpacked/dynamically generated code, thereby providing rich cross-platform and system-wide analysis capabilities.

Fig. 14 shows the overhead introduced by API Tracer on the execution of a Windows XP SP3 guest as the plugin scales with the number of functions in the plugin's configuration file.⁶ DECAF selectively instruments only the TCG TBs that correspond to the hooked functions, thereby significantly improving performance. An un-optimized implementation would instrument *all* TBs and filter the ones that correspond to hooks—similar to what TEMU [10] does. As a comparison, Internet Explorer loads the webpage (www.gnu.org) in 22.6 seconds and 217.79 seconds with selective optimization on and off, respectively. For the sake of evaluation, we considered two popular web browser clients for Windows-IE and Chrome, and a notorious bot TDSS [40] that inserts a kernel module to hide itself in the kernel. API Tracer is not only able to trace the inserted kernel module, but is also able to extract the unpacked code in memory for further analysis. The Chrome browser uses a *multiple-processes architecture* and keeps tabs, extensions, web apps, and plug-in processes independent from each other and spawns new processes when required. API Tracer is able to automatically trace the parent Chrome process and any spawned child processes.

7.4 Keylogger Detector

The Keylogger Detector plugin is an extended version of the sample plugin shown in Fig. 2. Leveraging the VMI, tainting, and event-driven programming features of DECAF, this plugin is capable of identifying keyloggers and analyzing their stealthy behaviors. The core of Keylogger Detector is cross-platform and OS-independent, comprised of only 120 lines of C code.

6. Configuration file consists of all functions that must be captured, along with their parameter list/types, return types, and calling conventions.

TABLE 7
Trojan.Win32.KeyLogger Trace

PROCESS	MODULE	FUNCTION
<KERNEL>	i8042prt.sys	hal.dll:READ_PORT_UCHAR
<KERNEL>	win32k.sys	ntoskrnl.exe:PsGetProcessWin32Process
<KERNEL>	win32k.sys	hal.dll:HalEndSystemInterrupt
...	...	...
notepad.exe	Mpk.dll	ntoskrnl.exe:ProbeForWrite
notepad.exe	Mpk.dll	user32.dll:SendMessageA
...	...	...
MPK.exe	user32.dll	ntoskrnl.exe:ProbeForWrite
...	...	...
MPK.exe	MPK.exe	kernel32.dll:InterlockedIncrement
...	...	...
MPK.exe	MPK.exe	hal.dll:HalEndSystemInterrupt
MPK.exe	MPK.exe	ntdll.dll:wscpy
MPK.exe	user32.dll	ntoskrnl.exe:ProbeForWrite
...	...	...

By sending tainted keystrokes into the guest system and observing if any untrusted code modules access the tainted data, we can detect keylogging behavior. The sample plugin can introduce tainted keystrokes into the guest system and identify which modules read the tainted keystroke by registering DECAF_READ_TAINTMEM_CB and DECAF_KEYSTROKE_CB callback events. To capture the detailed stealthy behaviors, Keylogger Detector implements a shadow call stack by registering a DECAF_BLOCK_END callback. Whenever the callback is triggered, we check the current instruction. If it is a *call* instruction, we retrieve the function information using VMI and push the current program counter onto the shadow call stack. If it is a *ret* instruction and pairs with the entry on the top of the shadow call stack, we pop it from the stack. When the DECAF_READ_TAINTMEM_CB callback is invoked, we retrieve information about which process, module, and function read the tainted keystroke data from the shadow call stack.

To evaluate our Keylogger Detector, we ran two kinds of experiments. First, we collected a set of malware samples that are known to have key-logging functionality. This sample set has 117 malware samples in total, spanning 29 malware families. We tested them on a Windows XP SP3 guest by sending keystrokes to the notepad application and observing whether any tainted keystrokes were accessed by the tested sample. Keylogger Detector successfully detected the keylogging behaviors in all of these samples. Table 7 is the trace of Trojan.Win32KeyLogger. It shows which module of the process read the tainted keystroke using which function. From the trace, we can tell that the tainted keystroke entered the system and was fetched by the untrusted code of MPK.exe, which clearly depicts a keylogging activity. Furthermore, the trace shows which functions were used to steal keystrokes. This information is very valuable when performing malware analysis.

Second, we created an analogous Keylogger Detector plugin for the TEMU tool and tested some tainted shell commands in both Windows XP Service Pack 3 and Linux 2.6.20 guests. We sent tainted keystrokes as commands to a shell and observed how each of the tainted commands was processed in the operating system. For each command, after it finishes execution, we observe the number of tainted bytes

TABLE 8
Comparing DECAF with TEMU on Tainted Shell Commands

Windows		
Command	DECAF	TEMU
dir	207 / 0	639 / 0
cd	146 / 0	616 / 0
cipher c:	929 / 0	3,617 / 0
echo hello	660 / 0	3,808 / 0
find "jone" a.txt	967 / 0	5,684 / 0
findstr /s /i jone ./*	945 / 0	1,333 / 0
Linux		
Command	DECAF	TEMU
ls	350 / 3	34,923 / 0
cd	306 / 3	301 / 0
cat ./readme	545 / 31	26,619 / 0
echo hello	744 / 9	704 / 0
ln -s a.txt nbench	1,122 / 35	24,707 / 0
mkdir test	551 / 9	23,766 / 0

"n / m" indicates that "n" bytes are tainted, and "m" tainted EIPs are observed.

in main memory and the occurrences of the EIP register becoming tainted. Note that, by design, the number of bytes tainted should be more correlated with the length of the commands than the actual commands used.

The results for both Windows and Linux are listed in Table 8. The results for Windows show that the number of tainted bytes in DECAF is much smaller than the number in TEMU, demonstrating the benefit of DECAF's tainting implementation being more precise. No instances of a tainted EIP register were observed in either system. The Linux results are somewhat different. Although the number of tainted bytes marked by DECAF was generally much smaller than that of TEMU, DECAF reported tainted EIP registers for all of the commands, whereas TEMU reported none.

These results look contradictory to the claim that DECAF should be more precise, so we manually examined the taint propagation logs generated by DECAF and TEMU. We did not examine every instances of a tainted EIP register (a total of 93), but we confirmed that every examined sample was indeed correct. A common case is that a tainted character (from the tainted keystroke we entered) was used as an index into a function pointer table to call a function. We found the same instruction sequences in the trace generated by TEMU. This means that TEMU has an under-tainting problem, even though its tainting rules are generally sound.

7.5 Instruction Tracer

The Instruction Tracer plugin records a TCG IR instruction-level trace with concrete and taint values for a specific guest user-space process or kernel code region. Similar to the other two plugins, Instruction Tracer is largely platform-neutral, capable of collecting execution traces for programs in x86 and ARM, Linux and Windows. Moreover, it is also easier to perform formal verification on the TCG trace, due

```

1. int func1(char *input) {
2.     char buffer[4];
3.     strcpy(buffer, input);
4. }
5. void main(void) {
6.     char buffer[16];
7.     scanf("%s", buffer);
8.     func1(buffer);
9. }
    
```

Fig. 15. A simple buffer overflow example.

```

qemu_ld32 tmp61[00000000],tmp50[00000000],$0x0
--> Taint HAS BEEN READ FROM MEMORY:
    Address: 0x07837e5c (4 bytes)
    Taint: [ffffffff]
movi_i32 tmp62[00000000],$0xffffffff
and_i32 pc[00000000],tmp61[00000000],tmp62[ffffffff]
--> Taint NOW PRESENT IN PROGRAM COUNTER (R15)
    
```

Fig. 16. Buffer overflow detection on ARM.

to its RISC-like instruction semantics, than on the original code of the guest. For example, it has been demonstrated as feasible to convert the TCG trace into LLVM IR and then perform symbolic execution on the trace [3]. Instruction Tracer is implemented in 3,860 lines of C code, though this includes the code for both the plugin and the parser for the log file that the plugin generates.

To demonstrate the practical effectiveness of this plugin, we used Instruction Tracer to detect a buffer overflow at runtime. The sample code in Fig. 15 was compiled and executed inside of x86 and ARM Linux guest VMs running under DECAF with Instruction Tracer loaded.

The code contains a simple buffer overflow vulnerability. If more than three characters are entered by the user, buffer in `func1()` will overflow and begin corrupting data stored on the stack. To capture the corruption, characters are entered into the program via tainted keypresses until the return address is modified by the overflow. Under the ARM environment, Instruction Tracer identified the buffer overflow when R15 (PC) became tainted after entering five characters. R14 (Link Register) was also monitored for taint, but it never became tainted during the test. Fig. 16 shows the log output at the point where R15 first becomes tainted. Tainted character data is fetched from stack memory, masked to ensure that the value is properly aligned, and then stored in R15.

Under the x86 environment, the TCG global variable for the EIP register can't be directly passed to an opcode as an argument. EIP is modified by writing to host memory via the `st_i32` opcode. Watching for tainted writes to EIP's offset (0x4C) in the `CPUState` data structure identifies that the buffer overflow. Fig. 17 shows the log output at the point where EIP first becomes tainted. Tainted character data is fetched from memory located at the address in ESP, the stack size is reduced by 4 bytes, and the tainted data is then placed into EIP's offset in the `CPUState` data structure.

We also performed a comparison of Instruction Tracer's performance against that of the TEMU's Tracecap plugin. Tracecap generates a trace of the guest's instructions as they execute to facilitate analyses similar to that of the buffer overflow analysis performed with Instruction Tracer. We used DECAF and TEMU to emulate the same Windows XP VM and trace the execution of an instance of the DOS sort

```

movi_i32 tmp2[00000000],esp[00000000]
qemu_ld32 tmp0[00000000],tmp2[00000000],$0x0
--> Taint HAS BEEN READ FROM MEMORY:
    Address: 0x0bffff30 (4 bytes)
    Taint: [ffffffff]
movi_i32 tmp15[00000000],$0x4
add_i32 tmp4[00000000],esp[00000000],tmp15[00000000]
mov_i32 esp[00000000],tmp4[00000000]
st_i32 tmp0[ffffffff],env,$0x4c
--> Taint NOW PRESENT IN EIP
    
```

Fig. 17. Buffer overflow detection on x86.

application. For both plugins, tainting was disabled. A text file 5.4 MB in size⁷ was selected to be sorted, and both plugins were configured to log their execution traces of the application directly to `/dev/null`. The `sort` execution completed in 39 m 57.33 s with Tracecap under TEMU, but in only 2 m 5.23 s with Instruction Tracer under DECAF (almost 20 times faster). The same `sort` with a stock QEMU completed in 5.89 s.

8 RELATED WORK

Several instrumentation solutions perform data flow analyses within the scope of a single process or binary. Such solutions are generally much faster than whole-system analysis platforms like DECAF, and they operate directly upon the native instructions of the binary under analysis. The Pin [1] API is a flexible C/C++ interface that is used to create instrumentation tools (known as “Pintools”). Examples of such Pintools are libdft [32] and DYTAN [14]. Pintools do not have the benefit of having a plugin development API that works at a semantic level higher than individual instructions, like DECAF does. DYTAN is designed as a platform for prototyping different tainting policies, while DECAF relies upon a proven sound and precise policy. libdft offers a less flexible, but faster, solution for tracking explicit data flows. It has the same limitations of other Pintools and, unlike DECAF, only supports instrumenting x86 binaries.

Many efforts have been made to reduce the runtime overhead of dynamic taint analysis. LIFT [16] assumes that taint propagation is not needed for most code execution, so it optimizes performance by taking the “fast paths” (without taint instrumentation) most of time. It also exploits extra registers in 64-bit architectures to shadow taints in 32-bit applications. Minemu [17] leverages the x86 SSE registers to provide lightweight taint tracking for 32-bit x86 applications. Jee et al. [41] build upon libdft to create a system that performs a static analysis on a process to selectively instrument the process for dynamic analysis per the rules of a Taint Flow Algebra. All of these tainting implementations only track taint status, and apply imprecise and sometimes unsound tainting rules, to achieve high efficiency. In comparison, DECAF is designed to perform accurate binary analysis in offline settings. So we cannot sacrifice precision and correctness for efficiency. DECAF is also designed to be generic, so we avoid relying on architecture-specific features (e.g., SSE) to boost up performance. Using static analysis to guide selective taint instrumentation is appealing, but is not generic and scalable in the whole-system setting.

Whole system instrumentation platforms leverage binary emulation and VMI. Early systems, such as TaintBochs [6], favor accuracy over performance. Ether [42] attempts to elude and analyze VM-aware malware by leveraging Intel VT hardware virtualization extensions. By triggering a debug exception after every instruction, Ether is able to fully analyze the state of the system, at the cost of heavy execution overhead. DECAF seeks to perform practical, accurate analyses of interactive systems, making the reduction of such high overhead a focus of its design.

7. The text file selected for this test was “The Complete Works of William Shakespeare”, which was downloaded from Project Gutenberg (<http://www.gutenberg.org>).

Like DECAF, TEMU [9] is built upon QEMU. It serves as the base for a variety of security analysis tools that perform whole-system analysis, such as HookFinder [7], Panorama [8], and Renovo [43]. TEMU is based upon version 0.9.1 of QEMU, which uses the older, defunct “dyngen” system (rather than TCG) for binary translation. TEMU is also not capable of emulating newer OSes such as Windows 7 and 8, and it is only capable of instrumenting x86 platforms. DECAF is capable of emulating these OSes and the ARM platform. S2E [3] uses QEMU to perform inline symbolic execution. When execution of the guest environment reaches a branch within code of interest, S2E forks the current QEMU process to explore both branches using symbolic execution. While powerful, this process is quite slow and memory intensive. DECAF is designed to assist in performing such heavyweight analyses by using lightweight plugins to capture detailed system information and instruction traces that provide enough detail to allow other tools to perform heavyweight analyses offline.

DroidScope [24] is a dynamic analysis platform for security analysis on Android. The core idea of DroidScope is to seamlessly reconstruct both Dalvik VM-level and OS-level semantic views and to provide a unified interface for Android malware analysis. DroidScope is an extension to DECAF for Android-specific analyses.

9 CONCLUSION

We present DECAF, a QEMU-based, multi-target, whole-system dynamic binary analysis framework. It implements a novel method of VMI and explicit sound and precise data flow tracking that incur much smaller runtime performance penalties than those seen in other whole-system analysis platforms. It provides a simple, event-driven plugin API for the development of largely platform-neutral analysis software.

ACKNOWLEDGMENTS

We would like to thank the anonymous reviewers for their comments. This research was supported in part by US National Science Foundation Grant #1018217, US National Science Foundation Grant #1054605, and McAfee Inc. Any opinions, findings, and conclusions made in this material are those of the authors and do not necessarily reflect the views of the funding agencies.

REFERENCES

- [1] C.-K. Luk, et al., “Pin: Building customized program analysis tools with dynamic instrumentation,” in *Proc. ACM SIGPLAN Conf. Program. Lang. Des. Implementation*, 2005, pp. 190–200.
- [2] N. Nethercote and J. Seward, “Valgrind: A framework for heavyweight dynamic binary instrumentation,” in *Proc. 28th ACM SIGPLAN Conf. Program. Lang. Des. Implementation*, 2007, pp. 89–100.
- [3] V. Chipounov, V. Kuznetsov, and G. Candea, “S2E: A platform for In-Vivo multi-path analysis of software systems,” in *Proc. 16th Int. Conf. Archit. Support Program. Lang. Oper. Syst.*, Mar. 2011, pp. 265–278.
- [4] G. Portokalidis, A. Slowinska, and H. Bos, “Argos: An emulator for fingerprinting zero-day attacks,” in *Proc. 1st ACM SIGOPS/EuroSys Eur. Conf. Comput. Syst.*, Apr. 2006, pp. 15–27.
- [5] J. R. Crandall and F. T. Chong, “Minos: Control data attack prevention orthogonal to memory model,” in *Proc. 37th Int. Symp. Microarchitecture*, Dec. 2004, pp. 221–232.

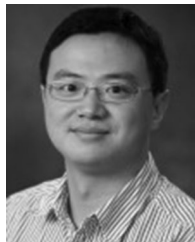
- [6] J. Chow, B. Pfaff, T. Garfinkel, K. Christopher, and M. Rosenblum, "Understanding data lifetime via whole system simulation," in *Proc. 13th USENIX Secur. Symp.*, Aug. 2004, pp. 22–22.
- [7] H. Yin, Z. Liang, and D. Song, "HookFinder: Identifying and understanding malware hooking behaviors," in *Proc. 15th Annu. Netw. Distrib. Syst. Secur. Symp.*, Feb. 2008, pp. 1–17.
- [8] H. Yin, D. Song, E. Manuel, C. Kruegel, and E. Kirda, "Panorama: Capturing system-wide information flow for malware detection and analysis," in *Proc. 14th ACM Conf. Comput. Commun. Secur.*, Oct. 2007, pp. 116–127.
- [9] TEMU: The BitBlaze dynamic analysis component. (2013). [Online]. Available: <http://bitblaze.cs.berkeley.edu/temu.html>
- [10] D. Song, et al., "BitBlaze: A new approach to computer security via binary analysis," in *Proc. 4th Int. Conf. Inf. Syst. Secur.*, 2008, pp. 1–25.
- [11] X. Jiang, X. Wang, and D. Xu, "Stealthy malware detection through VMM-based "out-of-the-box" semantic view reconstruction," in *Proc. 14th ACM Conf. Comput. Commun. Secur.*, Oct. 2007, pp. 128–138.
- [12] B. Dolan-Gavitt, T. Leek, M. Zhivich, J. Giffin, and W. Lee, "Virtuoso: Narrowing the semantic gap in virtual machine introspection," in *Proc. 2011 IEEE Symp. Secur. Priv.*, 2011, pp. 297–312. [Online]. Available: <http://dx.doi.org/10.1109/SP.2011.11>
- [13] Y. Fu and Z. Lin, "Space traveling across VM: Automatically bridging the semantic-gap in virtual machine introspection via online kernel data redirection," in *Proc. 2012 IEEE Symp. Secur. Privacy*, 2012, pp. 586–600.
- [14] J. Clause, W. Li, and A. Orso, "Dytan: A generic dynamic taint analysis framework," in *Proc. 2007 Int. Symp. Softw. Test. Anal.*, 2007, pp. 196–206.
- [15] J. Newsome and D. Song, "Dynamic taint analysis for automatic detection, analysis, and signature generation of exploits on commodity software," in *Proc. 12th Annu. Netw. Distrib. Syst. Secur. Symp.*, 2005, pp. 1–16.
- [16] F. Qin, C. Wang, Z. Li, H.-S. Kim, Y. Zhou, and Y. Wu, "LIFT: A low-overhead practical information flow tracking system for detecting security attacks," in *Proc. 39th Annu. IEEE/ACM Int. Symp. Microarchitecture*, 2006, pp. 135–148. [Online]. Available: <http://ieeexplore.ieee.org/lpdocs/epic03/wrapper.htm?arnumber=4041842>
- [17] E. Bosman, A. Slowinska, and H. Bos, "Minemu: The world's fastest tainttracker," in *Recent Advances in Intrusion Detection*. Berlin, Germany: Springer, 2011.
- [18] DECAF Binary Analysis Platform— "Taking the jitters out of dynamic binary analysis". (2014). [Online]. Available: <https://github.com/sycurelab/decaf/>
- [19] A. Alwabel, H. Shi, G. Bartlett, and J. Mirkovic, "Safe and automated live malware experimentation on public testbeds," in *Proc. 7th Workshop Cyber Secur. Exp. Test*, 2014. [Online]. Available: <http://www.usenix.org/conference/cset14/workshop-program/presentation/alwabel>
- [20] Q. Feng, A. Prakash, H. Yin, and Z. Lin, "MACE: High-coverage and robust memory analysis for commodity operating systems," in *Proc. 30th Annu. Comput. Secur. Appl. Conf.*, 2014, pp. 196–205.
- [21] C. Carmony, M. Zhang, X. Hu, A. V. Bhaskar, and H. Yin, "Extract me if you can: Abusing PDF parsers in malware detectors," in *Proc. Netw. Distrib. Syst. Secur. Symp.*, 2016, pp. 1–15.
- [22] J. Wei, L. K. Yan, and M. A. Hakim, "MOSE: Live migration based on-the-fly software emulation," in *Proc. 31st Annu. Comput. Secur. Appl. Conf.*, 2015, pp. 221–230.
- [23] F. Bellard, "QEMU, a fast and portable dynamic translator," in *Proc. USENIX 2005 Annu. Tech. Conf.*, 2005, pp. 41–41.
- [24] L. K. Yan and H. Yin, "DroidScope: Seamlessly reconstructing the OS and Dalvik semantic views for dynamic Android malware analysis," in *Proc. 21st USENIX Secur. Symp.*, 2012, pp. 29–29.
- [25] J. A. Goguen and J. Meseguer, "Security policies and security models," in *Proc. IEEE Comput. Soc. Symp. Res. Secur. Privacy*, May, 1982, pp. 11–20.
- [26] J. Seward and N. Nethercote, "Using Valgrind to detect undefined value errors with bit-precision," in *Proc. Annu. Conf. USENIX Annu. Tech. Conf.*, 2005, pp. 2–2. [Online]. Available: <http://dl.acm.org/citation.cfm?id=1247360.1247362>
- [27] Valgrind: Project suggestions, Jul. 2012. [Online]. Available: <http://valgrind.org/help/projects.html>
- [28] L. K. Yan, A. Henderson, X. Hu, H. Yin, and S. McCamant, "On soundness and precision of dynamic taint analysis," *Dep. Elect. Eng. Comput. Sci., Syracuse Univ., Tech. Rep. SYR-EECS-2014-04*, 2014.
- [29] C. Barrett, A. Stump, and C. Tinelli, "The SMT-LIB Standard: Version 2.0," presented at the 8th Int. Workshop Satisfiability Modulo Theories, Edinburgh, UK, 2010.
- [30] P. Godefroid and A. Taly, "Automated synthesis of symbolic instruction encodings from I/O samples," in *Proc. 33rd ACM SIGPLAN Conf. Program. Lang. Des. Implementation*, 2012, pp. 441–452. [Online]. Available: <http://doi.acm.org/10.1145/2254064.2254116>
- [31] D. Brumley, I. Jager, T. Avgerinos, and E. J. Schwartz, "BAP: A binary analysis platform," in *Proc. 23rd Int. Conf. Comput. Aided Verification*, Berlin, Heidelberg, 2011, pp. 463–469.
- [32] V. P. Kemerlis, G. Portokalidis, K. Jee, and A. D. Keromytis, "libdft: Practical dynamic data flow tracking for commodity systems," in *Proc. 8th ACM SIGPLAN/SIGOPS Conf. Virtual Execution Environments*, 2012, pp. 121–132. [Online]. Available: <http://dl.acm.org/citation.cfm?doid=2151024.2151042>
- [33] H. Yin and D. Song, "Temu: Binary code analysis via whole-system layered annotated execution," EECS Dep., Univ. California, Berkeley, USA, Tech. Rep. UCB/EECS-2010-3, Jan. 2010. [Online]. Available: <http://www.eecs.berkeley.edu/Pubs/TechRpts/2010/EECS-2010-3.html>
- [34] L. De Moura and N. Bjørner, "Z3: An efficient SMT solver," in *Proc. Theory Pract. Softw. 14th Int. Conf. Tools Algorithms Constr. Anal. Syst.*, Berlin, Heidelberg, 2008, pp. 337–340. [Online]. Available: <http://dl.acm.org/citation.cfm?id=1792734.1792766>
- [35] J. Ferrante, K. J. Ottenstein, and J. D. Warren, "The program dependence graph and its use in optimization," *ACM Trans. Program. Lang. Syst.*, vol. 9, no. 3, pp. 319–349, Jul. 1987. [Online]. Available: <http://doi.acm.org/10.1145/24039.24041>
- [36] M. G. Kang, S. McCamant, P. Poosankam, and D. Song, "DTA++: Dynamic taint analysis with targeted control-flow propagation," in *Proc. 18th Annu. Netw. Distrib. Syst. Secur. Symp.*, San Diego, CA, Feb. 2011, pp. 1–14.
- [37] J. Elgaard, N. Klarlund, and A. Møller, "MONA 1.x: New techniques for WS1S and WS2S," in *Proc. 10th Int. Conf. Comput.-Aided Verif.*, vol. 1427, Springer-Verlag, Jun./Jul.1998, pp. 516–520.
- [38] L. Martignoni, S. McCamant, P. Poosankam, D. Song, and P. Maniatis, "Path-exploration lifting: Hi-Fi tests for Lo-Fi emulators," in *Proc. 17th Int. Conf. Archit. Support Program. Lang. Oper. Syst.*, New York, NY, USA, 2012, pp. 337–348. [Online]. Available: <http://doi.acm.org/10.1145/2150976.2151012>
- [39] V. Ganesh and D. L. Dill, "A decision procedure for bit-vectors and arrays," in *Proc. 19th Int. Conf. Comput. Aided Verif.*, Berlin, Heidelberg, 2007, pp. 519–531.
- [40] S. Golovanov, "Analysis of TDSS rootkit technologies," *Securelist*, 2010. [Online]. Available: <http://www.securelist.com/en/analysis/204792131>
- [41] K. Jee, G. Portokalidis, V. P. Kemerlis, S. Ghosh, D. I. August, and A. D. Keromytis, "A general approach for efficiently accelerating software-based dynamic data flow tracking on commodity hardware," in *Proc. Netw. Distrib. Syst. Secur. Symp.*, 2012, pp. 1–17.
- [42] A. Dinaburg, P. Royal, M. Sharif, and W. Lee, "Ether: Malware analysis via hardware virtualization extensions," in *Proc. 15th ACM Conf. Comput. Commun. Secur.*, 2008, pp. 51–62.
- [43] M. G. Kang, P. Poosankam, and H. Yin, "Renovo: A hidden code extractor for packed executables," in *Proc. 5th ACM Workshop Recurring Malcode*, Oct. 2007, pp. 46–53.

Andrew Henderson is a fifth year PhD student at Syracuse University, Syracuse, New York. His research interests include dynamic binary code analysis, whole-system emulation and instrumentation, and system security. He is a senior member of the IEEE.

Lok Kwong Yan received the PhD degree in computer and information science and engineering from Syracuse University in 2013. He is a research scientist in Rome Air Force Research Lab, Rome, New York. His research interests include hardware security, dynamic binary code analysis, and malware detection and analysis. He is a senior member of the IEEE.



Xunchao Hu is a fourth year PhD student at Syracuse University, Syracuse, New York. His research interests include system security with a focus on program analysis, exploit diagnosis, and mobile security.



Heng Yin received the PhD degree in computer science from the College of William and Mary in 2009. He is an assistant professor in the Department of Electrical Engineering and Computer Science, Syracuse University, Syracuse, New York. His research interests lie in binary code analysis, mobile system security, virtualization, etc. He is a member of the IEEE.

Aravind Prakash is an assistant professor in the Department of Computer Science, State University of New York, Binghamton, New York. His research interest includes system security with a focus on program analysis, memory forensics, exploit diagnosis, and mobile security.



Stephen McCamant is an assistant professor in the Department of Computer Science and Engineering, University of Minnesota (Minneapolis). His primary research interest is applications of program analysis techniques for software security and correctness. This includes binary analysis and transformation, hybrids of dynamic and static analysis including symbolic execution, information flow and taint analysis, instruction-level hardening and isolation, and applications of decision procedures and proof-assistant tools. He is a member of the IEEE.

▷ For more information on this or any other computing topic, please visit our Digital Library at www.computer.org/publications/dlib.